Attack-Adaptive Network Intrusion Detection Systems for IoT Networks through Class Incremental Learning

Francesco Cerasuolo, Giampaolo Bovenzi, Domenico Ciuonzo, Antonio Pescapè

University of Napoli Federico II, Italy

Abstract

The advent of the Internet of Things (IoT) has ushered in an era of unprecedented connectivity and convenience, enabling everyday objects to gather and share data autonomously, revolutionizing industries, and improving quality of life. However, this interconnected landscape poses cybersecurity challenges, as the expanded attack surface exposes vulnerabilities ripe for exploitation by malicious actors. The surge in network attacks targeting IoT devices underscores the urgency for *robust* and *evolving* security measures. Class Incremental Learning (CIL) emerges as a dynamic strategy to address these challenges, empowering Machine Learning (ML) and Deep Learning (DL) models to adapt to evolving threats while maintaining proficiency in detecting known ones. In the context of IoT security, characterized by the constant emergence of novel attack types, CIL offers a powerful means to enhance Network Intrusion Detection Systems (NIDS) resilience and network security. This paper aims to investigate how CIL methods can support the evolution of NIDS within IoT networks (*i*) *by evaluating both attack detection and classification tasks*—optimizing hyperparameters associated with the incremental update or to the traffic input definition—and (*ii*) *by addressing also key research questions related to real-world NIDS challenges*—such as the explainability of decisions, the robustness to perturbation of traffic inputs, and scenarios with a scarcity of new-attack samples. Leveraging *4 recently-collected and comprehensive IoT attack datasets*, the study aims to evaluate the effectiveness of CIL techniques in classifying 0-day attacks.

1. Introduction

The rise of the Internet of Things (IoT) has marked an era characterized by unparalleled connectivity and convenience, wherein everyday objects can now autonomously collect and exchange data [1]. This paradigm shift has revolutionized industries, improved our overall quality of life, and opened up new frontiers in technology.

Nevertheless, this interconnected ecosystem introduces several cybersecurity challenges by expanding the attack surface and exposing vulnerabilities that malicious actors are eager to exploit. Alongside the rapid proliferation of IoT devices, there has been a noticeable surge in network attacks specifically targeting these devices [2, 3]. Cyber attackers acknowledge the potential for causing disruptions, stealing data, and gaining unauthorized access that arises from vulnerabilities in IoT systems. Consequently, the need for robust and adaptable security measures in the IoT landscape has never been more emphasized [4]. This challenge is exacerbated by the ever-evolving nature of the cybersecurity landscape, where defenders continuously fortify digital fortresses, while adversaries relentlessly search for vulnerabilities to exploit, even generating new kinds of attacks (viz. 0-*day attacks*) [5, 6].

In response to this dynamic scenario, Class Incremental Learning (CIL) emerges as a dynamic and adaptive strategy to tackle the challenge at hand. This approach empowers Machine Learning (ML) and Deep Learning (DL) models *to learn and adapt to new types of attacks* while retaining proficiency in detecting previously encountered threats [7]. In the realm of IoT security—characterized by the continuous emergence of novel attack types, diversity and low-cost nature of devices—CIL *can prove* to be a powerful means to enhance the resilience of Network Intrusion Detection Systems (NIDS) and bolster the security of network environments. Specifically, *CIL eliminates the need to train classifiers from scratch when new attack types occur, making them adaptive and thereby reducing the time required to deploy a fully-operational NIDS* capable of distinguishing among all attacks.

Accordingly, the contribution of this paper is twofold: (*i*) we investigate the incremental update of NIDS through CIL, with the aim of including knowledge of 0-day attacks, and (*ii*) we analyze incremental NIDSs when facing real-world challenges. These challenges include *opaque decision-making* in ML/DL architectures impeding their widespread adoption, *assessment of the degree of robustness* in the presence of slightly different network deployments or mutated/unforeseen network conditions, *scarce availability of samples for* 0-*day attacks to be included* requiring significant resources and specialized (or emulated) configurations.

Specifically, we address the following **research questions** associated with NIDS challenges in real-world scenarios:

- **Q1:** Can the knowledge of an existing (IoT-focused) NIDS be *extended to new attack traffic classes* by leveraging (ad-

Email addresses: francesco.cerasuolo@unina.it (Francesco Cerasuolo), giampaolo.bovenzi@unina.it (Giampaolo Bovenzi), domenico.ciuonzo@unina.it (Domenico Ciuonzo), pescape@unina.it (Antonio Pescapè)

vanced) CIL approaches? How are attack detection and classification capabilities separately affected?

- **Q2:** Can the optimization of hyperparameters, either associated to the incremental update (i.e. memory size) or to the traffic input definition (i.e. number of packets and selected packet fields as features) *improve the performance* of an extended NIDS?
- **Q3:** Can the *explainability of decisions* motivate the decision-process of incremental models?
- **Q4:** Can the *robustness to perturbation of inputs* be affected by the CIL approach leveraged to extend the NIDS?
- **Q5:** How do CIL approaches behave when there is (reasonably) *scarcity of new attack samples*?

To address these questions, *we leverage* 4 *of the most up-to-date and comprehensive IoT attack datasets currently available*, namely `IoT-23` [8], `TON-IoT` [9], `IoT-NID` [10], and `Edge-IIoT` [11], ensuring robust evaluation when investigating how CIL techniques perform in addressing the challenges of classifying 0-day attacks.

The rest of the paper is organized as follows. In the upcoming sections, we first delve into the literature related to the application of CIL approaches to attack classification (AC) and binary misuse detection (bMD) tasks (Sec. 2). Following that, we provide a comprehensive overview of our research methodologies (Sec. 3) and outline our experimental setup (Sec. 4). Then, we present our empirical findings (Sec. 5) and evaluate the proposed CIL-enabled NIDS methodology in the context of real-world challenges (Sec. 6). Last, we delve into discussions regarding the broader implications and future directions of our research (Sec. 7). For the sake of readability, the acronyms used in this manuscript are summarized in Tab. 1.

Table 1: List of the acronyms used in the manuscript.

Acronym	Definition
IoT	Internet of Things
CIL	Class Incremental Learning
ML	Machine Learning
DL	Deep Learning
NIDS	Network Intrusion Detection Systems
AC	Attack Classification
bMD	Binary Misuse Detection
FSCIL	Few-Shot Class Incremental Learning
MetaL	Meta-Learning
MG	Model Growth
FT	Fine-Tuning
AD	Anomaly Detection
TO	Traffic Object
CNN	Convolutional Neural Network
LSTM	Long Short-Term Memory
SOTA	State-of-the-art
BL	Baselines
SHAP	SHapley Additive exPlanations
IG	Integrated Gradients
BA	Balanced Accuracy
ROC	Receiver Operating Characteristic
AUC	Area Under Curve
PAUC	Partial Area Under Curve
FPR	False Positive Rate
PL	Network-layer Packet Length
IAT	Inter-Arrival Time
WIN	TCP Window Size
DIR	Packet Direction
TTL	Time-To-Live
FLG	TCP Flags

2. Related Work

In this section, we discuss literature concerning CIL in the domain of NIDS, with a specific focus on the challenge of *adapting existing classifiers/detectors to accommodate emerging types of attacks or patterns of anomalies*. CIL offers an advantage by continuously expanding models to address evolving threats while preserving previously acquired knowledge. Our literature review identified various approaches in this context, listed in Tab. 2 and categorized along different relevant axes.

While all the works under consideration explore incremental learning setups, only our preliminary conference paper [23] explores the adoption of incremental approaches to IoT traffic (column "**IoT**"). Specifically (column "**Dataset**"), most of them leverage the `KDDCUP99` dataset [12, 13, 14, 15, 16] or its enhanced version, `NSL-KDD` [7, 17]. Conversely, others employ `CIC-IDS2017` [18, 19, 20, 21, 22] dataset and or its improved version `CIC-IDS2018` [21]. In contrast to the aforementioned works, only Cerasuolo et al. [23] relies on the `IoT-23` dataset, a recently collected dataset encompassing traffic from real IoT devices.

Furthermore, considered works deal with different incremental tasks (columns "**Incremental Task**"). Notably, a few of them address the challenge of Few-Shot Class Incremental Learning (FSCIL) [7, 19, 21] by expanding the classifier while considering constraints of a limited number of samples. They analyze a different number of samples (viz. shots), specifically $\{3, 6, 10, 15\}$ in [7], $\{2, 5, 10\}$ in [19], $\{5\}$ in [21]. Conversely, the others tackle the incremental task without any constraint. Regarding the family of incremental approaches employed, for FSCIL tasks they employ meta-learning (MetaL) [7, 19], model-growth (MG) [7] or fine-tuning (FT) [21]. However, for CIL, they leverage only MG [22], or FT approaches [23].

In terms of the tasks addressed (column "**Task**"), the majority focus on distinguishing benign and malicious traffic in an unsupervised [12]—viz. *anomaly detection* (AD)—or supervised fashion [13, 14, 15, 16, 17]—viz. *binary misuse detection* (bMD)—while the remaining ones deal with classifying (in a flat fashion) between benign and all the attack classes—viz. *multiclass misuse detection* (mMD).

As for traffic object (column "**TO**"), all the considered work leverage *biflows* (viz. bi-directional flows) except for [19] considering (unidirectional) *flows* and [22] employing *sessions*. Interestingly, the majority of reviewed studies propose non-Deep Learning (DL) solutions (column "**DL**"). These solutions encompass a range of techniques (column "**ML/DL Approach**"), including clustering [12, 13], tree-based algorithms [14, 18], support vector machines [15, 16, 17, 18], k-nearest neighbor [14, 16, 7], Bayesian classifiers [7], and ensemble ap-

Table 2: State-of-the-art works in chronological order leveraging CIL to incrementally extend NIDS.

Paper	Year	IoT	Dataset	Incremental Task				Task	TO	DL	Early Detection	Raw Input	ML/DL Approach	Analyses		
				CIL	FSCIL	#Shots (K)	Appr. Family							Hyp.	XAI	Rob.
Burbeck et al. [12]	2007	✗	KDDCUP99	✓	✗	–	–	AD	B	✗	✗	✗	Clustering	○	○	○
Zhong et al. [13]	2008	✗	KDDCUP99	✓	✗	–	–	bMD	B	✗	✗	✗	Clustering	○	○	○
Yu et al. [14]	2009	✗	KDDCUP99	✓	✗	–	–	bMD	B	✗	✗	✗	K-Means, DT	○	○	○
Yi et al. [15]	2011	✗	KDDCUP99	✓	✗	–	–	mMD	B	✗	✗	✗	SVM	◐	○	○
Xu et al. [16]	2017	✗	KDDCUP99	✓	✗	–	–	mMD	B	✗	✗	✗	kNN-SVM	○	○	○
Constantinides et al. [17]	2019	✗	NLS-KDD	✓	✗	–	–	mMD	B	✗	✗	✗	SVM	◐	○	○
Wang et al. [7]	2021	✗	NLS-KDD	✗	✓	3, 6, 10, 15	MetaL MG	mMD	B	✓	✗	✗	DNN, KNN, SVM, BC	○	◐	○
Data et al. [18]	2022	✗	CIC-IDS2017	✓	✗	–	–	mMD	B	✗	✗	✗	AdaBoost, HT	○	○	○
Xu et al. [19]	2022	✗	CIC-IDS2017	✗	✓	2, 5, 10	MetaL	mMD	F	✓	✓	✓	1D-CNN	○	◐	○
Diallo et al. [20]	2023	✗	CIC-IDS2017	✓	✗	–	–	bMD, mMD	B	✓	✗	✓	Clustering	○	○	○
Du et al. [21]	2023	✗	CIC-IDS2017 CIC-IDS2018	✗	✓	5	FT	bMD, mMD	B	✓	✓	✓	Transformer	○	◐	○
Song et al. [22]	2023	✗	CIC-IDS2017	✓	✗	–	MG	mMD	S	✓	✓	✓	LSTM	○	◐	○
Cerasuolo et al. [23]	2023	✓	IoT-23	✓	✗	–	FT	bMD, mMD	B	✓	✓	✓	2D-CNN	○	◐	○
This paper	2024	✓	IoT-23, Edge-IIoT, TON-IoT, IoT-NID	✓	✓	5, 25, 50, 100, 200	FT	bMD, AC	B	✓	✓	✓	2D-CNN	●	●	●

Incremental Task: Class Incremental Learning (CIL), Few-Shot Class Incremental Learning (FSCIL); **Incremental Approach Family:** Meta-Learning (MetaL), Model-Growth (MG), Fine-Tuning (FT); **Task:** Attack Classification (AC), Anomaly Detection (AD), binary Misuse Detection (bMD), multi-class Misuse Detection (mMD); **Traffic Object (TO):** Biflow (B), Flow (F), Session (S); **ML/DL Approach:** Decision Tree (DT), Support Vector Machine (SVM), k-Nearest Neighbour (k-NN), Deep Neural Network (DNN), Bayesian Classifier (BC), Hoeffding Tree (HT), Convolutional Neural Network (CNN), Long Short-Term Memory (LSTM); **Hyper-parameters (Hyp.)**; **eXplainable AI (XAI)**; **Robustness (Rob.)**; ● performed, ◐ partially performed, ○ not performed; ✓ present, ✗ lacking.

proaches [18]. Conversely, other examined works leverage DL-based approaches [7, 19, 20, 21, 22, 23]. Specifically, they employ convolutional neural networks (CNNs) [19, 23], transformers [21], Long Short-Term Memory (LSTM) [22], or use DL feature extractors with clustering algorithms [20].

Crucially, most of these studies primarily concentrate on *post-mortem detection*—i.e. they do not perform *early detection* (column "**Early Detection**"). Specifically, these works leverage mainly statistics computed on the entire traffic object or, in the case of Diallo and Patras [20], they also employ header fields taken from *all* its packets. Conversely, other works only use fields extracted by packets header or payload as model input without relying on any summarizing feature (column "**Raw Input**"). The latter corpus of literature is usually matched with an early detection, as only the fields of the first N_p packets are used.

Another aspect to consider is the depth of analysis carried out by each study. Only a few works investigate how to optimize hyperparameters for enhanced performance (column "**Hyp.**"), such as examining the variations in SVM kernels [15] or scrutinizing the influence of adjusting the number of previous episode samples during the update phase [17]. Other studies delve into improving the explainability of incremental classifiers (column "**XAI**") only in a partial fashion. Specifically, they mainly leverage a single technique to determine the contribution to the performance of input or part of the design. For instance, Cerasuolo et al. [23] employ a feature-attribution technique (viz. Deep SHAP) to pinpoint the underlying causes behind misclassifications of specific class samples by the incremental classifier, while Xu et al. [19] and Du et al. [21] conduct ablation studies to gauge the effectiveness of individual components within their proposed solutions. Conversely, Song et al. [22] propose an explainable-by-design architecture and a corresponding ad-hoc explainability method is investigated to analyze feature importance for attack classification, defining a feature ranking. Finally, up to our knowledge, *no work deals with robustness analysis of CIL methods in the context of NIDS design* (column "**Rob.**").

Positioning of our work. In this study, our primary goal is to evaluate how CIL solutions enhance the adaptability of NIDS for IoT networks, enabling swift responses to emerging and previously unknown attacks (viz. 0-day). Accordingly, our study diverges from most state-of-the-art (SOTA) works as *we focus on IoT device traffic, encompassing 4 IoT datasets with diverse devices and attack scenarios* [8, 9, 10, 11].

Equally important, we employ cutting-edge CIL solutions to handle the complex challenges associated with NIDS design *which can be virtually applied to any DL model*. Specifically, *the considered CIL solutions belong to the fine-tuning family*, due to the higher complexity involved with using model-growth and meta-learning approaches which may impair their application to the IoT ecosystem [24, 25].

Furthermore, in stark contrast to *all* revised studies, our evaluation focuses on *how CIL solutions* empowering NIDS design *affect both AC and bMD tasks*. The aim is to assess the effect of the incremental training process on *NIDS (attack) detection and classification* capabilities in a *separate fashion*.

Also, as opposed to SOTA (focusing only on some model-dependent [15] or limited [17] aspects), we undertake a *comprehensive examination adjusting critical parameters* throughout the incremental process to optimize the performance of the adaptive NIDS.

Last but not least, we investigate *real-world challenges for incremental NIDS* including (*a*) explainability, (*b*) robustness, and (*c*) few-samples availability. Specifically, we leverage different input attribution techniques (i.e. SHAP and Integrated Gradients) to explain the reason behind predictions and we evaluate the accordance between them, we analyze the robustness of the incremental model's performance in presence of data corruption, and we evaluate performance when only a few-samples of 0-day attack are available for training models. Hence, our study represents a first attempt from the standpoint of (*b*) and a first general methodology for (*a*)—since [22] focused on explainability-by-design of a very specific DL model.

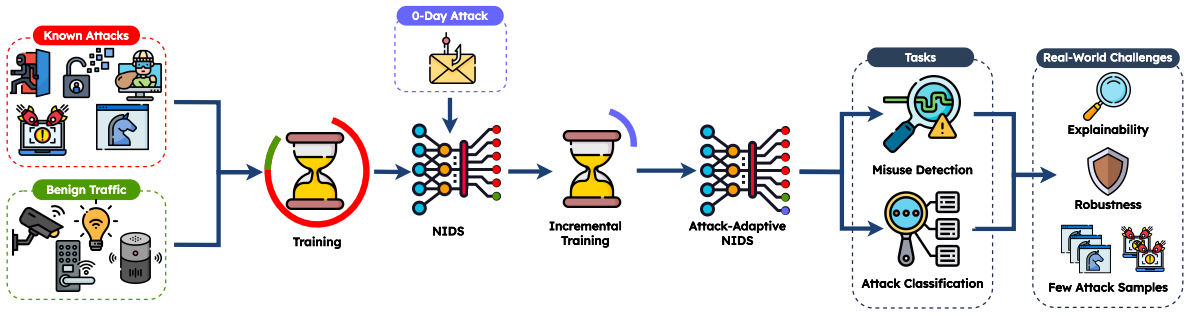


Figure 1: Overview of the process of incremental training of an Attack-Adaptive NIDS, including performed tasks and real-world challenges addressed.

Table 3: CIL approaches belonging to the *fine-tuning* family adopted in this work. They are categorized into Baselines (BL) and SOTA. For each approach, we reported the paper introducing that approach, employed mitigation strategies, and optimizable hyperparameters.

	Approach	Year	Mitigation Strategy				Hyper Params
			Rehearsal	Distillation	Regularization	Bias Correction	
BL	FT [26]	2020	○	○	○	○	–
	FT-Mem [24]	2023	●	○	○	○	mem-size
SOTA	LwF [27]	2017	○	●	●	○	λ_{dist}, λ_{reg}, T
	EWC [28]	2017	●	○	●	○	mem-size, λ_{reg}
	EEIL [29]	2018	●	●	○	◐	mem-size, T
	BiC [30]	2019	●	●	○	●	mem-size, T
	LUCIR [31]	2019	●	●	●	◐	mem-size, k, ρ_{base}, σ
	iCaRL+ [32]	2021	●	●	○	○	mem-size

Hyper Params: Memory size (mem-size), Temperature (T); ● present, ◐ partial/form of, ○ lacking;

3. Methodology

In the following section, we begin by presenting fundamental concepts concerning CIL (Sec. 3.1). Then, we delve into SOTA incremental approaches employed to design NIDS (Sec. 3.2). Next, we delineate the methodology utilized for the XAI analyses conducted (Sec. 3.3), as well as for the robustness analyses (Sec. 3.4) and few-shot scenarios (Sec. 3.5). An overview of the incremental training process (and corresponding real-world evaluation) for an attack-adaptive NIDS is depicted in Fig. 1.

3.1. Background on CIL

CIL represents a learning paradigm employed to update pre-trained models, enabling them to *incorporate new classes while retaining previously acquired knowledge*. This approach eliminates the need for retraining models from scratch and proves particularly advantageous in dynamic environments [24].

In detail, the CIL problem involves introducing new classes (C^{new}) leveraging associated data ($\mathcal{D}^{new}$) into a model initially trained on samples from old classes ($\mathcal{D}^{old}$) to expand its classification capabilities across the overall set of classes ($C^{all} = C^{old} \cup C^{new}$). This process leverages existing model parameters ($\boldsymbol{\theta}^*$), new classes data, and optionally withholds some old data as memory ($\mathcal{D}^{mem}$). In contrast to training-from-scratch (which uses $\mathcal{D} = \mathcal{D}^{new} \oplus \mathcal{D}^{old}$), the resulting model update leverages $\mathcal{D} = \mathcal{D}^{new} \oplus \mathcal{D}^{mem}$ as the training set, with $\mathcal{D}^{mem} \ll \mathcal{D}^{old}$.

Typically, a DL model consists of a *backbone*, responsible for feature extraction (i.e. $\boldsymbol{x} \rightarrow \boldsymbol{\varphi}(\boldsymbol{x})$) from the traffic input $\boldsymbol{x}$, and a *head*, dealing with classification and providing logits (i.e. the model output $\boldsymbol{\varphi} \rightarrow \boldsymbol{o}(\boldsymbol{\varphi})$). It is worth noting that logits have a size equal the number of classes to discriminate from. The logits are finally passed through a *softmax* layer (i.e. $\boldsymbol{o} \rightarrow \boldsymbol{p} = \text{softmax}(\boldsymbol{o})$) to provide a probabilistic confidence vector $\boldsymbol{p}$ associated with the classification task.

CIL methods fall into *three families* based on how they expand the model and which model parts they update [33]: (*i*) *fine-tuning* expands the model head and updates the entire model with new and *optionally* old data; (*ii*) *fixed-representation* expands the model head, and freezes parts of the pre-trained model (viz. backbone and/or old head), definitively training the non-frozen parts; (*iii*) *model-growth* maintains the base model and adds a new model (or trainable layers) for the new classes, combining their outputs for classification.

CIL encounters challenges such as *catastrophic forgetting* (loss of old knowledge) and *intransigence* (inability to learn new classes). To address these issues, mitigation strategies are introduced, including rehearsal (storing a small part of old data, with herding being the most adopted sampling strategy [34]), implicit- (namely distillation [27]) or explicit-regularization (minimizing updates to backbone parameters [28]), and bias correction (addressing model bias towards new classes [30]).

The training process is guided by a *composite loss function*, represented as

$$\mathcal{L} = \lambda_{class} \cdot \mathcal{L}_{class} + \lambda_{reg} \cdot \mathcal{L}_{reg} + \lambda_{dist} \cdot \mathcal{L}_{dist} \qquad (1)$$

This composite loss aims to simultaneously encourage model stability, through the regularization terms (i.e., $\mathcal{L}_{reg}$ and $\mathcal{L}_{dist}$), and plasticity, via the classification term (i.e., $\mathcal{L}_{class}$). The goal is to strike a balance between learning new information and retaining previously acquired knowledge.

In summary, CIL proves to be a valuable paradigm for adapting models to new data in evolving environments, eliminating the need for time-consuming retraining from scratch. In our specific context, our objective is *to enhance a classifier initially trained on a set of existing attacks and benign traffic* (viz. old classes) *by incorporating new knowledge about* 0*-day attacks* (viz. new classes).

3.2. CIL approaches

In this section, we outline the CIL approaches investigated in this work. Remarkably, all the methods utilized belong to the *fine-tuning* family [33], being recognized for achieving the highest performance in the mobile traffic classification context [24] and shown to be promising in preliminary attempts towards adaptive NIDS design [23]. Notably, we opt not to explore *model-growth approaches* due to their limited scalability with increasing increments and higher memory demands, making them unsuitable for deployment in the IoT context.

Initially, we introduce *two* baseline (BL) methods, namely `FT` and `FT-Mem`, representing the most straightforward approaches. Subsequently, we delve into *six* more advanced methods that integrate one or more mitigation strategies recalled in Sec. 3.1. Table 3 offers a comprehensive overview of the considered approaches, along with an indication of the employed mitigation strategies for each.

`FT`**—Fine-Tuning:** it represents the most straightforward solution for model updates, involving the update of the entire model with only the samples of new classes (viz. $\mathcal{D}^{new}$).

`FT-Mem`**—Fine-Tuning with Memory:** a variation of the previous approach, `FT-Mem` withholds a subset of old classes samples as a memory. These retained samples are reintroduced into the model during the incremental training to cope with catastrophic forgetting (viz. $\mathcal{D} = \mathcal{D}^{new} \oplus \mathcal{D}^{mem}$).

`LwF`**—Learning Without Forgetting** [27]: this is a *memory-free* approach (viz. $\mathcal{D} = \mathcal{D}^{new}$) that pioneers the application of Knowledge Distillation (KD) in incremental learning scenarios. KD improves generalization performance by transferring expertise from a pre-trained model (viz. teacher) to a target network (viz. student) [35]. Specifically, `LwF` employs a three-term loss: a classification term ($\mathcal{L}_{class}$) dedicated to incorporating new classes, distillation ($\mathcal{L}_{dist}$) and regularization ($\mathcal{L}_{reg}$) terms to retain previously acquired knowledge of old classes. The balance among classification, distillation, and regularization losses is upheld by adjusting hyperparameters λ_{dist} and λ_{reg}[1], which are specifically applied to the distillation and regularization terms, respectively.

`EWC`**—Elastic Weights Consolidation** [28]: this approach relies on the Fisher information matrix (FIM) diagonal, indicating the sensitivity of each parameter's loss derivative near a minimum. High values imply significant parameter influence on the loss, necessitating stabilization to prevent catastrophic forgetting. `EWC` uses a single head, freezing old parameters during updates. It employs a two-term loss function: $\mathcal{L}_{class}$, akin to `LwF`'s approach but applied only to new classes, and a regularization term $\mathcal{L}_{reg}$ that weighs parameter importance based on the FIM. This regularization balances adaptation to new classes while preserving old class parameters. In its original formulation, `EWC` computes the FIM post-training solely from $\mathcal{D}^{new}$. However, integrating a memory, as suggested by [33], enhances performance, especially in CIL scenarios. Therefore, the algorithm first updates the model and then, as post-processing steps (i.e. needed for subsequent updates), it computes the Fisher information matrix and herding selection.

`EEIL`**—End-to-End Incremental Learning** [29]: this approach aims to enhance `iCaRL`'s design [34] by integrating *end-to-end* learning mechanisms that concurrently update the classifier and the feature representation. `EEIL` utilizes a dual-term loss function comprising a classification loss term ($\mathcal{L}_{class}$) and a distillation term ($\mathcal{L}_{dist}$), similar to that of `LwF`. Additionally, `EEIL` introduces *balanced fine-tuning* to mitigate the inherent imbalance between new samples and old class exemplars ($|\mathcal{D}^{new}| \gg |\mathcal{D}^{mem}|$). Following the model update, `EEIL` selects a subset of $\mathcal{D}^{new}$ equal in size to the memory and conducts a secondary fine-tuning step using the same procedure outlined previously but with a reduced learning rate. As a result, `EEIL` operates a follows. First, distillation probabilities are collected, then the model update is triggered. The model update is then refined via balanced fine-tuning. Finally, herding selection is performed to build memory for next updates.

`LUCIR`**—Learning a Unified Classifier Incrementally via Rebalancing** [31]: this method introduces a sophisticated regularization approach, surpassing `EWC`'s focus on the backbone and `EEIL`'s treatment of magnitude bias. The loss function includes $\mathcal{L}_{class}$, akin to `EEIL`'s, and two additional terms, $\mathcal{L}_{dist}$ and $\mathcal{L}_{reg}$. Specifically, $\mathcal{L}_{dist}$ is weighted by $\rho_{base} \cdot \sqrt{|C^{new}|/|C^{old}|}$ (ρ_{base} being a dataset-specific parameter) while for the other two terms $\lambda_{reg} = \lambda_{class} = 1$. First, $\mathcal{L}_{class}$ is applied to all classes (old + new) with softmax computed on *cosine-distance normalized* scores $\overline{o}_i(\boldsymbol{x}) \triangleq \text{cd}(\boldsymbol{\zeta}_i, \boldsymbol{\varphi}(\boldsymbol{x}))$, where $\boldsymbol{\zeta}_i$ is the ith *class embedding*[2] and $\text{cd}(\boldsymbol{v_1}, \boldsymbol{v_2}) \triangleq \langle \boldsymbol{v_1}, \boldsymbol{v_2} \rangle / (\|\boldsymbol{v_1}\| \, \|\boldsymbol{v_2}\|)$ is the cosine distance. The model head of `LUCIR` projects the feature vector $\boldsymbol{\varphi}(\boldsymbol{x})$ into a cosine space ($\in [-1, 1]$), potentially alleviating *magnitude bias*. A learned parameter η automatically scales all $\overline{o}_i(\boldsymbol{x})$'s to compensate for magnitude differences across updates. Additionally, the *less-forget loss* distillation $\mathcal{L}_{dist}$, computed with the old and updated feature vectors ($\boldsymbol{\varphi}^*(\boldsymbol{x})$ and $\boldsymbol{\varphi}(\boldsymbol{x})$, respectively), is used to constrain the latent space. This loss term is computed as $\mathcal{L}_{dist} = \sum_{x \in \mathcal{D}} (1 - \text{cd}(\boldsymbol{\varphi}^*(\boldsymbol{x}), \boldsymbol{\varphi}(\boldsymbol{x})))$. Furthermore, a *marginal-rank loss* reinforces the separation between old and new classes, contrasting class embeddings' reactions when activated by old class exemplars $\boldsymbol{x} \in \mathcal{D}^{mem}$—the cosine-normalized score of the *true* old class $\bar{o}_{gt}(\boldsymbol{x})$ should be higher than the same measure applied to any of the new classes, i.e. $\bar{o}_i(\boldsymbol{x}), \forall i \in C^{new}$. This loss compares $\overline{o}_{gt}(\boldsymbol{x})$ against the top-k measures from new classes, $\mathcal{T}_k$, adding a penalty only if the difference is smaller than a margin σ: $\mathcal{L}_{reg} = \sum_{\boldsymbol{x} \in \mathcal{D}^{mem}} \left\{ \sum_{i \in \mathcal{T}_k} \max\left(0, \sigma - \bar{o}_{gt}(\boldsymbol{x}) + \bar{o}_i(\boldsymbol{x})\right) \right\}$.

[1] In `LwF`, λ_{reg} was consistently set as a fixed parameter at $5 \cdot 10^{-4}$.

[2] `LUCIR` head uses embeddings rather than a linear layer head.

`iCaRL+`**—Incremental Classifier and Representation Learning Plus** [32]: this approach is an adapted and improved version of `iCaRL` [34] for the TC domain. `iCaRL+` includes a rehearsal strategy (via herding selection), leverages an output layer that dynamically expands and activates through the softmax function (in contrast to the Nearest Mean Classifier employed by `iCaRL`), and employs a loss function comprising classification and distillation (KD) loss terms, each equally weighted ($\lambda_{class} = \lambda_{dist} = 1$).

`BiC`**—Bias Correction** [30]: this method tackles the bias toward new classes by introducing an *additional layer to correct* the bias of the model head output. This bias correction layer is parameterized by two factors α and β. The correction is specifically applied to the logits of new classes ($\boldsymbol{o}^{new}$), while the logits of old classes ($\boldsymbol{o}^{old}$) remain unchanged. Precisely, the correction transforms $\boldsymbol{o}^{new}$ in $\bar{\boldsymbol{o}}(\boldsymbol{x}) = \alpha \cdot \boldsymbol{o}^{new}(\boldsymbol{x}) + \beta \cdot \mathbf{1}$. The incremental training process for this approach then operates as follows. First, to reduce forgetting, the model is updated using a composite loss function with classification and KD terms (weighted by $\lambda_{class} = |C^{new}|/|C^{all}|$ and $\lambda_{dist} = 1 - \lambda_{class}$, respectively). Then, the additional layer is trained with a small dataset to determine the two correction parameters by *freezing both the model components* (viz. backbone and head). Specifically, the dataset to train the correction layer encompasses a balanced number of per-class samples, obtained as $\lceil \mathcal{P}\% \cdot N_{old} \rceil$, where N_{old} represents the number of old samples for each class belonging to C^{old}. In the original formulation and previous works, $\mathcal{P}$ is set to 10% of per-class memory samples.

For completeness, in our analysis, we consider the model trained from scratch with all data from both old and new classes as the performance upper bound. Henceforth, we will refer to it as `Scratch`.

3.3. XAI Methodology

To interpret complex DL architectures after incremental training, we leverage *input-attribution* methods providing a *local explanation* (viz. per-sample explanation) to elucidate the model behavior in the neighborhood of each instance ($\boldsymbol{x}$) after model training (viz. *post-hoc* analysis). Per-sample explanations from local methods are here aggregated to obtain *global explanations*. In this work, we leverage *SHapley Additive exPlanations* (SHAP) [36] and *Integrated Gradients* (IG) [37].

SHAP. Shapley values, originating from cooperative game theory, determine each player's contribution m to the overall payoff $v(P)$ of the entire coalition P. In this method, subsets of cooperating players $S \subset P$ are assessed to gauge the impact of adding or removing player m on the total payoff $v(S)$. When interpreting DL-based models, input data serve as players in the cooperative game, and the DL architecture's output $f(\boldsymbol{x})$ acts as the payoff function.

In this work, we employ DeepSHAP [38], a rapid approximation algorithm, to gauge the significance of inputs in determining the class (i.e. *SHAP values*). Specifically, we interpret SHAP values (e_m) as the importance of inputs in determining confidence levels for class labeling.

Integrated Gradients. IG is based on the input baseline concept, derived from human tendencies to attribute effects through counterfactual intuition. It implicitly compares a specific effect, like a classifier decision, with its absence, achieved by modeling the absence using a single baseline input.

In essence, IG quantifies the importance of each input (e_m) in altering confidence relative to the same class with the baseline input. It achieves this by considering a straight-line path from the baseline ($\boldsymbol{x}'$) to the input ($\boldsymbol{x}$), computing gradients at all points along this path, and then cumulating them.

Lastly, we normalize SHAP/IG *local explanations* ($\widetilde{e}_m = e_m / \sum_{m=1}^{M} e_m$) and then aggregate them across diverse samples obtaining *global explanations* [39]. In this work, we leverage SHAP and IG to explain confidence levels associated with predicted classes (i.e. legitimate traffic or specific attacks).

Furthermore, given the absence of a standardized XAI framework and the potential for feature-attribution methods to yield varied explanations, leveraging diverse techniques and quantifying their *coherence* is crucial for accurate interpretation [40].

Coherence of Explanations. We evaluate the *coherence* of different explanation techniques through two different per-sample metrics, namely $Top5Ratio$ and $Avg.Rel.$ [41]. The former measures the similarity between explanations considering the top-5 most important features for different explanation methods. It is computed as follows:

$$Top5Ratio = \frac{|Top_5(SHAP(\boldsymbol{x})) \cap Top_5(IG(\boldsymbol{x}))|}{5}, \quad (2)$$

where $SHAP(\boldsymbol{x})$ (resp. $IG(\boldsymbol{x})$) refers to the explanation obtained by DeepSHAP (resp. IG). The $Top5Ratio$ metric is then normalized within $(0, 100)\%$, where 100% signifies full agreement among XAI techniques whereas 0% indicates total disagreement.

Conversely, the $Avg.Rel.$ quantifies the overall difference in input importance attributed to different methods to the same input biflow. It is computed as:

$$Avg.Rel. = \frac{1}{Z} \sum_{i=1}^{Z} |SHAP_i(\boldsymbol{x}) - IG_i(\boldsymbol{x})|, \quad (3)$$

where Z denotes the number of input components (i.e. the size of $\boldsymbol{x}$). Accordingly, a smaller $Avg.Rel.$ suggests greater consistency, indicating their similarity, while a larger value implies lower reliability.

Then, we obtain a *global* version of these two metrics averaging each across all considered samples.

3.4. Robustness to Perturbed Inputs

In this work, we analyze the robustness of the incremental and `Scratch` models to input perturbations, generally described as $\boldsymbol{x} \rightarrow \boldsymbol{\Delta}(\boldsymbol{x})$. Specifically, we assess whether and how much traffic input perturbations *seen only during the inference stage* affect NIDS detection and classification performance. We purposefully refrain from delving into adversarial perturbations [42], as they tend to be model-specific and are crafted

based on the responses of a particular model. Instead, we opt to employ more universally-applicable techniques, leveraging general primitives $\mathbf{\Delta}(\cdot)$ to manipulate input data [43]. This broader approach allows us to explore perturbation methods that are not tied to specific models and can potentially have wider applicability across various domains and contexts. Equally important, we focus on input manipulations employed *which mirror real-world network scenarios*.

Specifically, we consider perturbations emulating modification in network conditions: (*i*) *jittering* that mimic fluctuations in IAT between consecutive packets

$$\mathbf{IAT}_{jit} \triangleq \begin{bmatrix} \text{iat}_1 & \cdots & \text{iat}_{N_p} \end{bmatrix} + \begin{bmatrix} \epsilon_1 & \cdots & \epsilon_{N_p} \end{bmatrix},$$

where ϵ_i ($i = 1, \ldots, N_P$) are different random values while N_P denotes the number of packets considered in the input; (*ii*) *timeout* emulating network overloading and packet loss leading to TCP packet retransmission; (*iii*) *flipping* that alter the direction of the communication—i.e. modifies the DIR of packets from upstream to downstream and vice-versa; (*iv*) *slicing* simulating an interruption of communication, hence NIDS can focus only on the first half of the biflow segment for decisions:

$$B_{slicing} \triangleq \begin{bmatrix} \text{pkt}_1 & \cdots & \text{pkt}_{\frac{N_p}{2}} \end{bmatrix} + \begin{bmatrix} \overline{\text{pkt}}_{\frac{N_p}{2}+1} & \cdots & \overline{\text{pkt}}_{N_p} \end{bmatrix},$$

where B and pkt denote the biflow and packet, respectively, while $\overline{pkt}$ the packet masked with padding values for each input field.

It is worth noticing that the first two perturbations are random, while the latter two are deterministic. Hence, in the former case, the (random) perturbation process relies on some distributional parameters. Specifically, the random values for *jittering* perturbation are drawn from a Gaussian distribution defined by the pair (μ, σ), whereas the *timeout* is triggered by a Bernoulli-sampled packet loss with probability ρ.

3.5. Few-shot scenarios methodology

In this work, we explore scenarios characterized by *limited data availability* for 0-day attacks, evaluating the impact of the number of samples available for incremental training on classification and detection performance.

This scenario is commonly known as FSCIL, as it intersects with both CIL and Few-Shot Learning (FSL). FSCIL involves enhancing a base classifier to accommodate the knowledge of new classes (C^{new}) using a notably restricted number of samples. Accordingly, the updated model is trained using $\mathcal{D} = \mathcal{D}^{mem} \cup \mathcal{D}^{few}$ as the training dataset, where $\mathcal{D}^{few} \ll \mathcal{D}^{new}$ (viz. $|\mathcal{D}^{few}| = K \cdot |C^{new}|$). The number of samples for each new class K is also referred to as "shots" in FSCIL literature. To recreate a few-shot scenario, we perform an incremental training procedure *across multiple episodes*, where each episode relies on a *different set* of K shots. These shots are randomly sampled (without replacement) from $\mathcal{D}^{new}$ to generate a distinct $\mathcal{D}^{few}$ for each episode. The latter set is used to learn the new apps (C^{new}) in a FSCIL context. As a result, multiple evaluation phases (corresponding to the number of episodes) are carried out on the same testing data for each incremental model in this context. Also, in what follows, we assess the impact across various shot values, notably $K = \{5, 25, 50, 100, 200\}$.

4. Experimental Setup

In this section, we present a thorough overview of the experimental setup utilized in our analyses. Specifically, Sec. 4.1 presents the metrics employed for evaluating CIL approaches in the context of NIDS design. The characteristics of leveraged IoT traffic datasets are discussed in Sec. 4.2, while Sec. 4.3 delineates the model architecture taken into consideration. Lastly, Sec. 4.4 introduces the relevant CIL setups and scenarios.

4.1. Evaluation metrics

In this study, we employ the *F1 Score* (briefly F1), a commonly used metric for assessing DL model performance and evaluating the **classification capabilities** of a CIL model. This metric represents the harmonic mean of precision and recall for each class, with the macro F1 being the arithmetic average over all classes, expressed as:

$$\text{F1}(\boldsymbol{\theta}, C) = \frac{1}{|C|} \sum_{i \in C} \frac{2 \cdot Precision_i \cdot Recall_i}{Precision_i + Recall_i}, \tag{4}$$

where C denotes a generic set of classes, and $\boldsymbol{\theta}$ represents a generic model, learned either from scratch or in an incremental manner.

Given that the F1, influenced by precision and recall, may obscure the model's behavior, this study also includes *Balanced Accuracy* (BA) to enhance evaluation and provide a clearer understanding of whether catastrophic forgetting or intransigence is occurring [44]. This metric is defined as follows:

$$BA(\boldsymbol{\theta}, C) = \frac{1}{|C|} \sum_{i \in C} Recall_i. \tag{5}$$

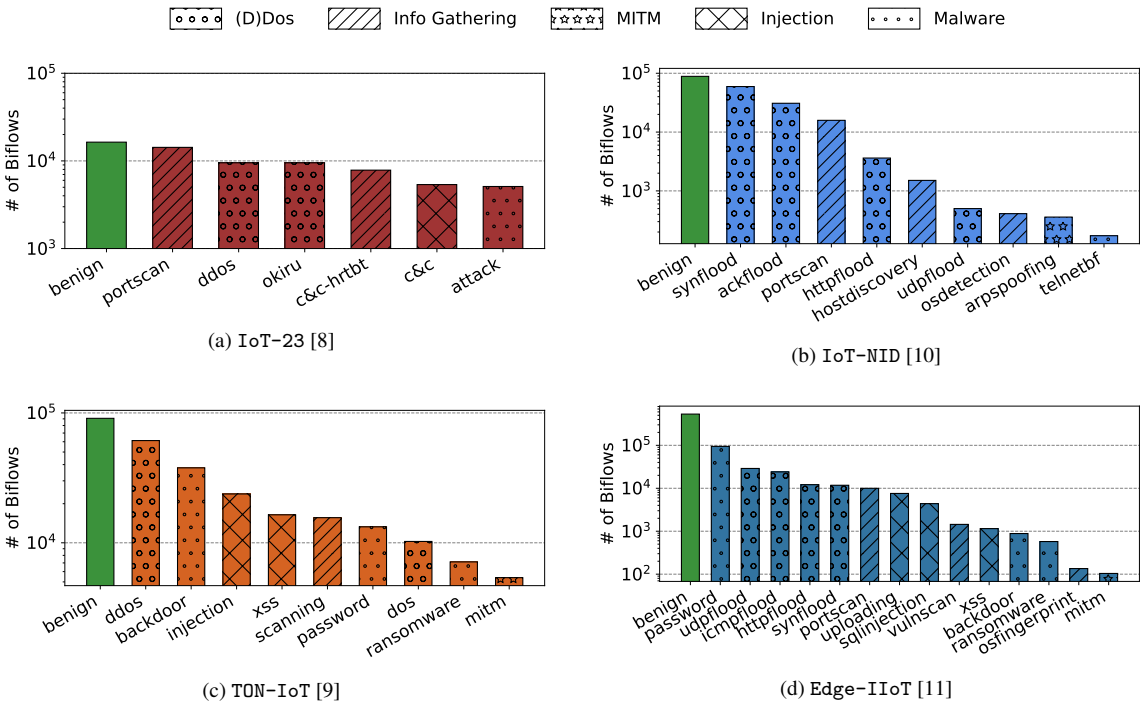
Accordingly, in this work F1 and BA are both used *to assess NIDS performance related to the AC task*.

Furthermore, we utilize the *Receiver Operating Characteristic* (ROC) curve and the *Area Under the ROC Curve* (AUC) to evaluate the **detection capabilities** of an incremental classifier. The ROC curve visually represents the performance of binary classification (viz. detection) models, illustrating how their performance changes with adjustments to detection thresholds. In this study, we use the sum of soft outputs associated with attack classes as the detection score. Instead, the AUC is a numeric measure summarizing the overall performance of a binary classification model by quantifying the area under the ROC curve. A perfect (or random) model has an AUC of 100% (or 50%). Higher AUC values indicate better discrimination between positive and negative cases. Additionally, we consider the *Partial Area Under the ROC Curve* (PAUC), focusing on the interval enclosed by a maximum False Positive Rate (FPR) value of 1% to evaluate model performance over a restricted range of FPRs in the ROC curve.

Specifically, in this work we resort on the *macro version of AUC and PAUC*. This allows us (*i*) computing the (P)AUC given an arbitrary set of attack classes w.r.t. the benign one, and (*ii*) avoiding the class-imbalance effect on such metrics. Accordingly, given an arbitrary set of malicious traffic classes—denoted as C_m—and the benign class c_b, the macro-(P)AUC is

Table 4: Categorization of attack classes within leveraged datasets, indicating the nature of devices used, including real (🖥) or emulated (🤖) devices.

Dataset	Period	Attack Category					Devices
		(D)DoS	*Info Gathering*	*MITM*	*Injection*	*Malware*	
`IoT-23` [8]	2018–19	**DDoS Okiru**	**C&C-Heartbeat Port Scanning**	N.A.	**C&C**	**Attack**	🖥
`IoT-NID` [10]	2019	**DoS SYN Flood Mirai ACK Flood Mirai HTTP Flood Mirai UDP Flood**	**Host Discovery OS/Version Scanning Port Scanning**	**ARP Spoofing**	N.A.	**Mirai Telnet Bruteforce**	🖥
`TON-IoT` [9]	2019-20	**DDoS DoS**	**Scanning**	**MITM**	**Injection XSS**	**Backdoor Password Ransomware**	🖥 🤖
`Edge-IIoT` [11]	2021–22	**DDoS HTTP Flood DDoS ICMP Flood DDoS TCP SYN Flood DDoS UDP Flood**	**OS Fingerprinting Port Scanning Vulnerability Scanner**	**MITM**	**SQL Injection Uploading XSS**	**Backdoor Password Ransomware**	🖥 🤖



(a) `IoT-23` [8]

(b) `IoT-NID` [10]

(c) `TON-IoT` [9]

(d) `Edge-IIoT` [11]

Figure 2: Biflows distribution of leveraged IoT datasets. Various hatch patterns are employed to distinguish between different attack categories.

defined as:

$$\text{macro-(P)AUC}(\boldsymbol{\theta}, C_m) = \frac{1}{|C_m|} \sum_{c_i \in C_m} \text{(P)AUC}(c_i, c_b) \quad (6)$$

Hereinafter, for brevity, we refer to the macro-(P)AUC simply as (P)AUC. In what follows, AUC and PAUC are are used *to assess NIDS performance associated with the bMD task*.

To gain deeper insights into the classification/detection performance gap between an incremental model and its counterpart trained-from-scratch, we introduce a *differential version* of each evaluation metric. For any arbitrary metric $\mathcal{X}$, its differential version, denoted as $\mathcal{X}$ Drop, is computed as follows:

$$\mathcal{X}\,\text{Drop}(C) = \mathcal{X}(\boldsymbol{\theta}^{Scratch}; C) - \mathcal{X}(\boldsymbol{\theta}^{CIL}; C) \quad (7)$$

This kind of metric is further broken down by sets of classes into $\mathcal{X}$ DropAll for C^{all}, $\mathcal{X}$ DropOld for C^{old}, and $\mathcal{X}$ DropNew for C^{new}, representing the drops associated with all, old, and new classes (attacks), respectively. *Lower values of these metrics indicate better performance*.

4.2. Datasets, Traffic Segmentation, and Input Selection

Taxonomy of IoT Traffic Datasets. In this study, we investigated 4 datasets comprising network traffic originating from IoT devices, collected from 2019 onwards [45]. They differ in the nature of the malicious traffic they comprise. In detail, `IoT-23` and `IoT-NID` encompass traffic generated by real devices, while `TON-IoT` and `Edge-IIoT` consist of traffic generated by both real and emulated devices. Furthermore, they incorporate a broad spectrum of attacks, where specific types are evident across multiple datasets, while others remain exclusive to particular datasets (see Tab. 4 for a detailed taxonomy). It is important to note that this classification of attack classes is further corroborated by prominent surveys in this domain [45, 46, 47]. Specifically, all datasets include *UDP*

and *SYN flooding* attacks, with *TCP flooding* attacks exclusively present in IoT-23 and TON-IoT. Furthermore, only IoT-NID includes *ACK flooding* attacks and Edge-IIoT exhibits *ICMP flooding* attacks, while both present *HTTP flooding* attacks. These attacks aim to disrupt service availability by inundating it with excessive traffic from one or multiple sources. The datasets cover various scanning attacks, including *port scanning* (in IoT-23, IoT-NID, TON-IoT, and Edge-IIoT), *OS scanning* (in IoT-NID and Edge-IIoT), *vulnerability scanning* attacks (in Edge-IIoT), and *host scanning* (in IoT-NID). Although these classes do not refer to actual attacks, they usually represent the reconnaissance phase that is preparatory for the exploitation of network attacks. Specifically, these (malicious) actions play a crucial role in identifying vulnerabilities and streamlining unauthorized access. Additionally, IoT-23 incorporates keep-alive attacks (viz. *C&C-heartbeat*) to exploit vulnerabilities. *Man-in-the-middle* attacks are observed, encompassing *ARP spoofing* in IoT-NID, TON-IoT, and Edge-IIoT. TON-IoT incorporates also *ICMP* and *DHCP spoofing*, while Edge-IIoT involves *DNS spoofing*. These attacks consist of unauthorized interception of communications between two parties. *SQL injection* and *XSS* are collected in TON-IoT and Edge-IIoT, and *HTTP injection* in Edge-IIoT, involving the unauthorized entry of code compromising system integrity. Furthermore, IoT-23 includes Command and Control (*C&C*) attacks, involving unauthorized infiltration to gain control over compromised systems for malicious activities. Other included attacks involve *brute-force* attempts on HTTP (IoT-23, TON-IoT, and Edge-IIoT), telnet (IoT-NID and IoT-23), FTP (TON-IoT), and SSH (IoT-23). These attacks systematically test various password combinations to gain unauthorized access. Finally, *backdoor* (unauthorized access points for remote entry) and *ransomware* (malicious software encrypting files) are present in TON-IoT and Edge-IIoT.

Concerning the devices utilized for dataset collection, IoT-23 and IoT-NID datasets employ real devices, encompassing both smart home and general-purpose devices. In contrast, Edge-IIoT and TON-IoT datasets leverage a combination of emulated and real devices, including sensors and general-purpose devices.

Traffic Object Definition. A commonly utilized traffic object for early detection is the biflow [48]. The latter is a *bidirectional flow*, encompassing packets that share an identical quintuple (*source IP*, *source port*, *destination IP*, *destination port*, and *transport protocol*), with the option of interchanging the *source* and *destination*. Based on the aforementioned traffic segmentation, the dataset compositions in terms of number of biflows for each class are illustrated in Figs. 2a–2d. In this visualization, the benign class is depicted in green, while the attack classes are differentiated by various colors, each corresponding to a specific dataset.

Definition of Traffic Input. In this study, we consider *four* per-packet properties, specifically the network-layer packet length (PL), inter-arrival time (IAT), TCP window size (WIN), and direction (DIR). Furthermore, we also examine the potential benefit of using additional inputs such as Time-To-Live (TTL) and TCP Flags (FLG). In detail, PL represents the size of the packet, IAT indicates the time gap between two consecutive packets, WIN denotes the TCP window size (with a value set to 0 for UDP packets), DIR signifies the direction of the packet (+1 for upstream, −1 for downstream), TTL is the maximum number of hop that a packet can traverse through a network before it is discarded, and FLG refer to control bits in the TCP header. We underline that certain fields like transport ports and IP addresses are intentionally omitted from our model's training to avoid undue bias [49].

Accordingly, in this work, we consider the per-packet properties of the first N_p packets to design an NIDS able to detect IoT-focused threats in an early fashion, i.e. achieving robust attack detection and classification performance without compromising the earliness of its outcomes. Consequently, we extract time series data for each biflow, encompassing a sequence length of N_p packets for the specified per-packet properties. If the time series data consists of less than N_p packets, we employ zero-padding to reach the necessary length. Previous studies [49, 50] have established that the initial $N_p = 10$ packets of each biflow are sufficient for accomplishing the aforementioned goal: hence, we put forward this design choice in the following experiments, while also providing a sensitivity analysis w.r.t. N_p.

4.3. DL Model Architecture

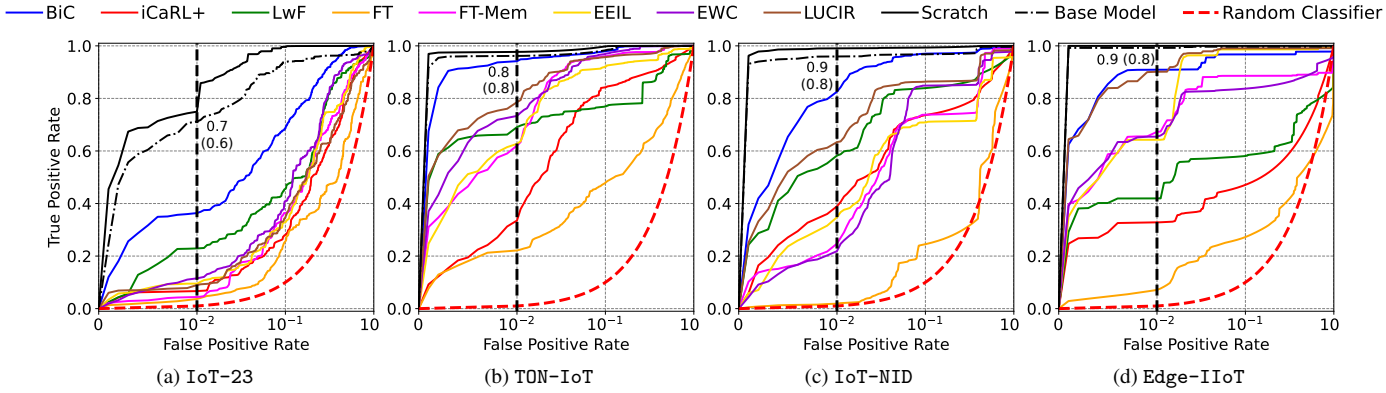
In this work, we leverage a 2D-CNN architecture, consisting of two convolutional layers (Conv2D), followed by pooling and normalization operations, and a Dense layer responsible for generating the model's output. It is noteworthy that CNN-based models find extensive application in SOTA approaches for traffic fingerprinting [51, 52]. The entire architecture involves approximately 540k trainable parameters.

The network is fed with 10×4 [*packets* $\times$ *fields*] per biflow as the input $\boldsymbol{x}$. As we have chosen to employ fine-tuning family approaches, the backbone remains unchanged throughout each incremental iteration, while the head (i.e. the final layer of the network) is expanded to accommodate as many neurons as the new classes introduced in that specific episode. Incremental training begins with the parameter values derived after the preceding training stage ($\boldsymbol{\theta}^*$).

It is worth noting that the proposed methodology is designed to be architecture-agnostic—i.e., while it has been applied to a specific DL architecture, it can be adapted for use across various other DL architectures. This flexibility enhances its applicability in diverse settings.

4.4. Class Incremental Learning Scenario

In this work, our main emphasis is on adapting NIDS to *new emerging attacks* targeting IoT devices. Specifically, we assume the availability of a base NIDS model able to discriminate between legitimate traffic and a number of known attacks. The objective is then to include into the model new attack classes upon their discovery in an *incremental* fashion. In our experimental investigations, we delve into a prevalent cyber-security scenario involving 0-*day* attacks. We focus on the scenario



(a) IoT-23 (b) TON-IoT (c) IoT-NID (d) Edge-IIoT

Figure 3: ROC curves averaged across increments for CIL approaches, Scratch, and base model. Curves are depicted with a linear scale from 0 to 10^{-2} and then with a *logarithmic* scale. Annotations report the TPR of C^{old} (C^{new}) for the base model obtained with FPR= 10^{-2}, showcasing its capability to detect known—viz. misuse detection—and 0-day attacks—viz. anomaly detection—, respectively.

Table 5: Detection performance in terms of PAUC Drop (lower values are the better) of CIL approaches and Base Model. Performance expressed as *average* and *std. dev.* across the runs. Furthermore, we present the PAUC values derived from training the model from scratch. Specifically, the obtained PAUC is ≈83% for IoT-23, ≈94% for TON-IoT, ≈91% for IoT-NID, and ≈99% for Edge-IIoT. The **best** (resp. **worst**) CIL approach is reported with **green** (resp. **red**) colored cell; the **best** approach overall is reported in **bold**.

Approach	IoT-23			TON-IoT			IoT-NID			Edge-IIoT		
	All	**Old**	**New**	**All**	**Old**	**New**	**All**	**Old**	**New**	**All**	**Old**	**New**
FT	31.55	31.72	30.71	37.55	39.00	25.94	39.83	39.89	39.30	47.39	47.42	47.09
FT-Mem	31.36	31.20	32.18	22.27	22.92	17.04	30.08	30.95	23.10	24.43	25.34	12.61
LwF	25.83	25.23	28.81	14.69	14.30	17.82	24.87	23.79	33.54	28.60	29.15	21.41
iCaRL+	29.65	31.24	21.69	34.22	35.24	26.12	31.17	32.00	24.51	35.04	35.18	33.16
LUCIR	28.91	28.19	32.51	12.07	10.96	20.89	25.89	24.89	33.92	11.22	11.36	9.43
EEIL	28.62	30.09	21.24	22.11	23.10	14.18	29.12	30.86	**15.19**	25.51	26.97	**6.47**
EWC	28.92	29.01	28.45	16.78	16.62	18.05	32.47	33.52	24.06	21.37	21.85	15.13
BiC	18.77	17.54	24.94	4.20	3.95	**6.22**	18.18	16.74	29.70	8.93	8.70	11.94
Base Model	**3.80**	**0.71**	**19.25**	**0.71**	**−0.04**	6.68	**2.02**	**−1.11**	27.01	**−0.05**	**−0.57**	6.70

where a new attack is added to the model. This approach is based on the reasonable assumption that multiple new attacks do not emerge simultaneously. Therefore, when samples of a novel attack are detected (and labeled as such), they are incrementally incorporated into the NIDS.

To match the aforementioned assumptions, for each considered dataset (consisting of N attack classes plus the benign class), our evaluation consists of N runs. Specifically, n^{th} run is obtained by excluding n^{th} attack from the base model training, and treating it as a 0-day. Hence, in each run a different attack is treated as new and added incrementally. Since this approach entails conducting N (incremental) experiments for a given dataset, the results are presented as the average and standard deviation of the metrics discussed in Sec. 4.1 across all the runs. Notably, for the FSCIL scenario (Sec. 3.5), results are expressed as average and standard deviation across different episodes (i.e. 10 in our setup) and across the N runs. Evaluation of each model follows the hold-out method, utilizing 70% and 30% of the dataset as the training and test sets, respectively.

The experiments being reported rely on the FACIL framework [33], employing the following training parameters: 200 epochs with a patience of 20, an initial learning rate of 0.1, a learning rate decay factor of 3, and a minimum learning rate of 10^{-4}. For memory-based CIL approaches, we set a memory composed of 25 exemplars per class—viz. memory size is obtained as $25 \times |\mathcal{C}|$. As for input perturbation, while *flipping* and *slicing* are static, *jittering* and *timeout* are parametrized. Notably, for *jittering*, we empirically set parameters ($\mu = 0$, $\sigma = 0.3$). Differently from Cerasuolo et al. [44], which introduced this transformation to augment traffic biflows preserving the biflow semantics (with $\mu = 0$, $\sigma = 0.03$), we choose $\sigma = 0.3$ to alter appreciably the temporal dynamics of biflows by introducing a delay in the reception of packets of ±8ms (viz. $\sigma = 4$ms). For the *timeout* perturbation, we have selected a packet loss rate of $\rho = 10\%$. Concerning the computational resources utilized for the experimental campaign, all analyses were performed on a machine equipped with 2 Intel(R) Xeon(R) Gold 6252N CPUs running at 2.30 GHz and boasting 378 GB of memory.

5. Experimental Results

In this section, we address the bMD task in Sec. 5.1 and then the incremental AC task in Sec. 5.2. Notably, we benchmarked several CIL approaches, namely FT, FT-Mem, LwF, iCaRL+, LUCIR, EEIL, EWC, and BiC, against the 4 IoT attack datasets considered: IoT-23, TON-IoT, IoT-NID, and Edge-IIoT.

Table 6: CIL approaches performance—in terms of F1 and BA Drop (lower values are the better)—and training time. Performance and time are expressed as *average* and *std. dev.* on the runs. Time saved is computed as percentage w.r.t. training time required by `Scratch`. Additionally, we specify the F1 (resp. BA) achieved by training the models from scratch. In particular, `IoT-23` attains ≈ 89% (resp. ≈ 90%), `TON-IoT` ≈ 98% (resp. ≈ 99%), `IoT-NID` ≈ 97% (resp. ≈ 96%), and `Edge-IIoT` ≈ 79% (resp. ≈ 80%). The **best** (resp. **worst**) CIL approach is reported with **green** (resp. **red**) colored cell.

Dataset	Approach	F1 Drop [%]			BA Drop [%]			Time Saved [%]
		All	Old	New	All	Old	New	
IoT-23	FT	81.30± 3.90	85.45± 4.33	60.51±11.06	75.13± 7.37	88.73± 3.09	20.77±47.43	87
	FT-Mem	35.92±14.26	36.46±14.85	33.22±13.86	35.97±19.69	40.21±14.76	19.01±45.17	85
	LwF	39.53±13.01	30.24±14.08	85.96± 9.73	34.19±12.05	26.50±16.49	64.94±29.36	81
	iCaRL+	29.96±24.98	30.01±28.35	29.75±11.21	29.23±27.51	33.70±25.77	11.36±38.82	79
	LUCIR	42.02±20.22	43.86±22.94	32.83±13.65	40.54±14.81	47.79±23.51	11.54±38.50	78
	EEIL	28.99±16.08	27.91±18.13	34.41±11.24	30.03±20.85	34.63±17.50	11.65±38.95	94
	EWC	20.62± 6.94	19.83± 7.53	24.55± 6.31	21.11±12.06	25.04±11.57	5.38±26.24	80
	BiC	10.92± 4.30	9.08± 4.64	20.13± 4.68	9.67± 7.08	10.70± 8.44	5.56±19.30	80
TON-IoT	FT	91.12± 3.74	92.72± 4.02	78.35±12.52	85.62± 6.70	94.87± 3.18	20.89±41.92	93
	FT-Mem	19.28± 6.46	16.68± 7.18	40.09±17.23	17.48± 7.80	19.34± 8.77	4.44±15.27	93
	LwF	33.81±15.02	25.79±16.91	97.95± 2.10	29.07±16.84	21.56±17.94	81.63±33.14	91
	iCaRL+	27.58±23.77	25.18±25.96	46.78±18.09	23.81±22.49	26.47±26.02	5.21±12.60	90
	LUCIR	13.75± 6.95	12.21± 7.18	26.04±23.02	12.61± 7.61	14.28± 8.55	0.92± 3.73	89
	EEIL	21.10± 7.85	18.85± 7.37	39.10±17.38	20.03± 6.39	22.43± 8.03	3.29± 8.60	97
	EWC	12.75± 5.22	10.20± 4.61	33.14±18.18	10.16± 3.59	11.56± 4.01	0.33± 3.87	90
	BiC	5.48± 4.92	3.57± 3.81	20.82±19.82	5.03± 4.72	4.09± 4.02	11.58±24.20	90
IoT-NID	FT	90.17± 4.53	91.43± 4.91	80.08±24.35	77.13± 5.72	88.68± 6.66	− 3.70± 4.37	96
	FT-Mem	28.84±25.38	25.96±27.64	51.87±27.66	20.25±23.71	23.64±26.9	− 3.51± 4.28	96
	LwF	47.12±26.75	40.85±29.91	97.22± 2.40	36.63±23.67	28.11±26.86	96.29± 4.36	94
	iCaRL+	37.08±31.65	35.28±35.17	51.48±25.87	26.61±31.18	30.90±35.42	− 3.41± 4.26	92
	LUCIR	32.40±19.51	29.57±21.50	55.01±27.91	25.16±16.89	28.90±19.74	− 1.08± 4.31	93
	EEIL	23.25±11.79	20.30±14.28	46.81±19.26	14.99±11.64	17.61±13.20	− 3.41± 4.26	97
	EWC	26.01±13.93	23.70±16.25	44.45±27.93	18.45±13.54	21.56±15.26	− 3.31± 4.23	93
	BiC	18.16±13.50	15.74±12.40	37.47±32.45	11.56±10.06	11.73±12.52	10.39±28.47	93
Edge-IIoT	FT	73.67± 7.46	74.34± 7.97	64.91±36.68	67.88± 8.39	73.97± 9.21	− 5.18±55.81	98
	FT-Mem	36.71±15.53	36.80±15.91	35.50±38.98	25.11±16.41	28.64±18.32	−17.27±36.99	97
	LwF	62.22± 9.00	60.91±10.18	79.29±33.50	54.27±15.22	52.66±15.91	73.51±40.74	98
	iCaRL+	60.69±23.68	61.07±23.40	55.76±44.70	54.41±26.57	59.35±27.27	− 4.89±55.73	96
	LUCIR	23.21±15.63	23.00±15.83	25.94±37.87	15.85±14.05	17.67±14.71	− 6.03±53.31	96
	EEIL	34.21±15.29	33.20±14.88	47.35±43.15	24.29±14.52	26.85±14.46	− 6.34±53.42	97
	EWC	23.25±15.79	23.26±16.81	23.14±31.47	14.79±16.62	17.11±18.56	−13.06±44.85	96
	BiC	12.85± 7.88	11.82± 7.47	26.14±24.42	9.76± 7.15	9.90± 8.31	8.16±38.39	96

Subsequently, we explore the hyperparameters crucial for incremental training—i.e. *number of memory biflows*—and for traffic input definition—i.e. *number of packets* and *set of feature used*—in Sec. 5.3. Finally, in Sec. 5.4, we determine the optimal configuration for each dataset and discuss the results.

5.1. Misuse Detection

Aim of the analysis: In this section, we investigate the performance of incremental NIDS, obtained with different CIL approaches, in distinguishing between legitimate and malicious traffic (viz. bMD task).

The results are depicted using the ROC curves in Fig. 3 and quantified through *PAUC Drop* in Tab. 5. It is worth noting that ROC curves are averaged across all increments, as detailed in Sec. 4.1. Additionally, we recall that *the detection score is computed as the sum of the soft outputs associated with attack classes*.

Across all examined datasets, the `FT` method, which is the most straightforward approach, turns out to be the worst performing approach in differentiating between benign and malicious classes, resulting in a PAUC Drop All of ≈ 32% for `IoT-23`, ≈ 38% for `TON-IoT`, ≈ 40% for `IoT-NID`, and ≈ 48% for `Edge-IIoT`. Due to pronounced forgetting, the performance on C^{old} consistently deteriorates, obtaining values of PAUC Drop ≥ 32%. Moreover, the same model encounters difficulty in distinguishing between C^{new} and the benign class, achieving drop values higher than ≈ 26%. These results underscore the necessity of implementing strategies to mitigate forgetting.

The adoption of memory is of clear benefit for bMD performance. Indeed, even `FT-Mem` attains a PAUC DropAll of ≈ 22% for `TON-IoT`, ≈ 30% for `IoT-NID`, and ≈ 24% for `Edge-IIoT`. Moreover, it substantially enhances performance across the Old classes, achieving ≈ 23% for `TON-IoT`, ≈ 31% for `IoT-NID`, and ≈ 25% for `Edge-IIoT` of PAUC Drop. Even within C^{new}, notable improvements are evident, with drop of ≈ 17% for `TON-IoT`, ≈ 23% for `IoT-NID`, and ≈ 13% for `Edge-IIoT`. Differently, for `IoT-23` the performance of `FT-Mem` is not too far from `FT`, reaching similar values for C^{all} and C^{old} and worsening

the drop value on C^{new}. This underlines the need for distilling the behavior of the base model other than simply replaying old class samples.

Furthermore, the sole addition of a regularization term (i.e. LwF) fails to yield improvement for Edge-IIoT but obtains better performance on the other datasets, reducing the drop for all classes to ≈ 25% for IoT-23, ≈ 15% for TON-IoT, and ≈ 25% for IoT-NID.

Combining mitigation strategies leads to improved performance for TON-IoT and Edge-IIoT. In detail, LUCIR outperforms previously mentioned approaches reaching a PAUC Drop All of ≈ 12% and ≈ 11%, respectively.

BiC emerges as the most effective CIL approach, demonstrating better detection performance w.r.t. other approaches. For instance, it achieves the lowest drop value on TON-IoT, reaching a ≈ 4% PAUC Drop All and Old, and ≈ 6% of PAUC Drop New.

Counter-intuitively, the base model achieves the most robust detection performance. Specifically, for all the datasets, it achieves better detection performance than Scratch for C^{all} and C^{old} presenting low PAUC Drop values, in some cases also <0. As for C^{new}, it presents PAUC Drop values >0 that results the best across all the considered approaches only for IoT-23 with a ≈ 19% PAUC Drop New. Conversely, for TON-IoT, it exhibits a drop of ≈ 7% for the new class and it is surpassed by BiC, obtaining a 1% lower drop (≈ 6%). Instead, for IoT-NID and Edge-IIoT, EEIL reaches the lowest PAUC Drop New with ≈ 15% and ≈ 6%, respectively, overperforming base model which obtains ≈ 27% and ≈ 7%. These performance outcomes may stem from the absence of *catastrophic forgetting* in the base model, hence it continues detecting old classes with satisfactory results while sufficiently distinguishing the new attack class from benign ones.

Take-home: BiC emerges as the most effective CIL approach to address the bMD task. However, **utilizing a (simple) base classifier may suffice for distinguishing between malicious and benign traffic (in contrast to the set of CIL techniques considered)**. This is particularly notable in IoT-23 while for the other three datasets, incremental approaches (viz. BiC or EEIL) lead to (*slightly*) better detection performance on the new class. This finding highlights that **cutting-edge incremental techniques beyond the current state-of-the-art are needed** to improve detection performance.

5.2. Attack Classification

Aim of the analysis: In this section, we evaluate the performance of the attack classification task, specifically focusing on the ability to distinguish between malicious (viz. attack) traffic classes. This includes assessing the classifier's effectiveness when a new previously unseen (i.e., 0-day) attack is introduced into the underlying NIDS base model.

According to the results presented in Tab. 6, FT attains the worst performance, showing significant drops across all class sets. Specifically, FT incurs an F1 Drop ≥ 74% for C^{all} and C^{old} and ≥ 61% for C^{new}. This result is consistent with the findings of the bMD task, suggesting that the absence of memory detrimentally affects performance in both tasks.

By implementing various mitigation strategies, the drop across different subsets of attacks tends to diminish. Specifically, FT-Mem (viz. FT plus rehearsal techniques) notably enhances FT's performance, resulting in a decrease of ≈ 36% on IoT-23, ≈ 19% on TON-IoT, ≈ 29% on IoT-NID, and ≈ 37% on Edge-IIoT. Solely relying on regularization techniques, such as LwF, hampers the model's ability to include new class knowledge, leading to a drop ≥ 79%. However, leveraging multiple strategies yields improved outcomes. For instance, the combination of strategies benefits IoT-23, where EWC exhibits a drop of ≈ 21% on C^{all}, ≈ 20% on C^{old}, and ≈ 25% on C^{new}. Similarly, in the case of TON-IoT, EWC demonstrates drops of ≈ 13%, ≈ 10%, and ≈ 33% on all, old, and new class subsets, respectively. Among the methods explored, EEIL exhibits the lowest drops for all and old class subsets on IoT-NID (≈ 23% and ≈ 20%, respectively), while EWC achieves the most favorable drop for new classes (≈ 44%). Finally, concerning Edge-IIoT, LUCIR obtains the lowest drops for all and old class subsets (≈ 23% for both), whereas EWC achieves the most promising result for new classes (≈ 23%).

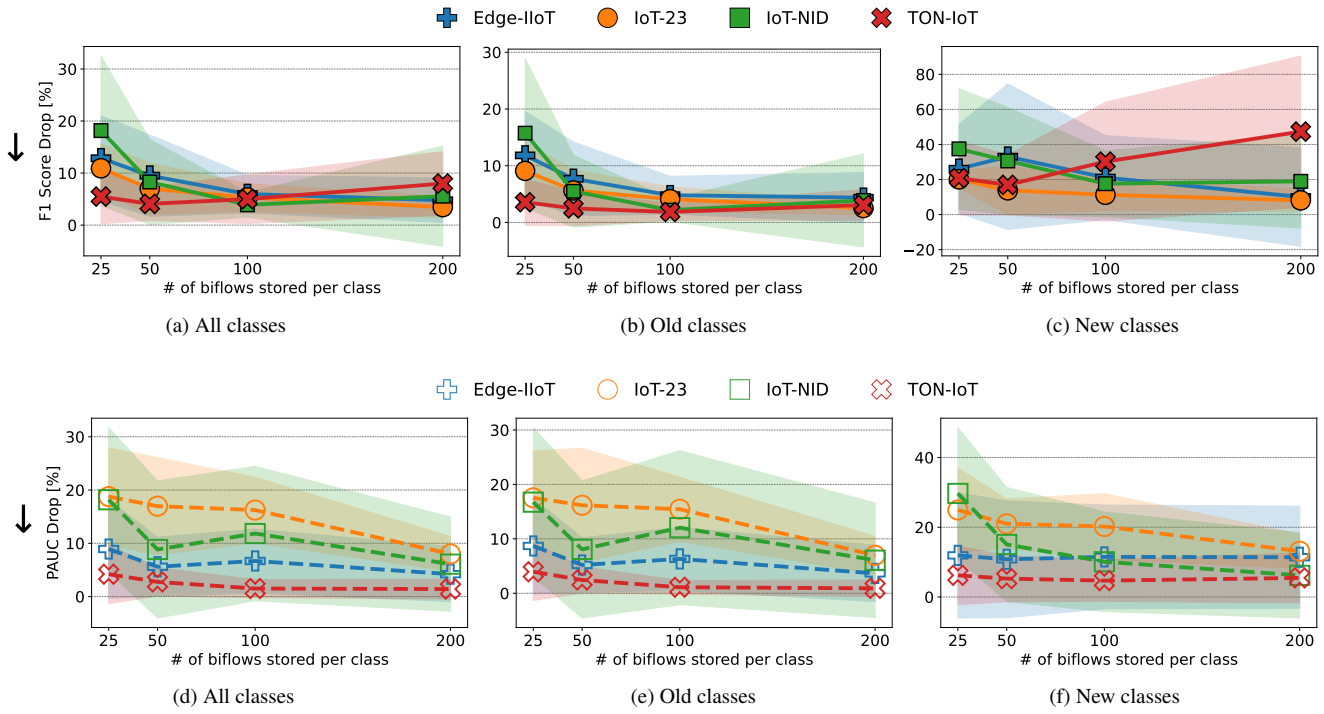
Finally, BiC emerged as the top performer across all datasets, showcasing the most favorable outcomes in terms of F1 DropAll. Notably, it achieved ≈ 11% on IoT-23, ≈ 5% on TON-IoT, ≈ 18% on IoT-NID, and ≈ 13% on Edge-IIoT. Furthermore, BiC obtains the lowest F1 DropOld values, at most below 4% on TON-IoT. However, it exhibited relatively higher F1 DropNew, ranging from 20% to 38%. Despite this, BiC maintained the lowest F1 DropNew for all datasets except Edge-IIoT, where EWC showcased slightly superior performance regarding C^{new}.

In addition, all the considered approaches provide substantial time savings (in the training phase) in comparison to Scratch. Notably, the EEIL approach stands out with the most significant time savings (except for Edge-IIoT) attributed to the loss function leading the incremental training process and to the early stopping procedure. The remaining approaches exhibit slightly lower time savings, influenced by both the loss function and the utilization of memory samples. Specifically, iCaRL+ offers the least time savings on IoT-23 (≈ 79%) and IoT-NID (≈ 92%). On TON-IoT, LUCIR turns out to be the most time-consuming approach (≈ 89%), whereas for Edge-IIoT, iCaRL+, LUCIR, EWC, and BiC require the same training time (96% saving w.r.t. Scratch). The remaining approaches offer time savings intermediate between FT and the most time-consuming approaches.

Take-home: Looking at Tab. 6, **memory-free approaches (i.e. FT and LwF) struggle to obtain satisfactory performance, while using a memory is more effective at handling the CIL scenario.** Furthermore, **the overall best-performing model is BiC** offering performance closest to Scratch (viz. upperbound) for all the considered datasets. Finally, **adopting BiC can lead to training time savings of up to** 80% **compared to Scratch on** IoT-23 (≈ 330s and ≈ 1800s, resp.).

5.3. Investigating (Class Incremental) Hyperparameters

Aim of the analysis: This section analyzes several key hyperparameters essential for the incremental model to improve classification and detection performance. Specifically, we focus on



(a) All classes (b) Old classes (c) New classes

(d) All classes (e) Old classes (f) New classes

Figure 4: Analyzing memory size impact on incremental model performance in terms of F1 (*a*, *b*, *c*) and PAUC Drop (*d*, *e*, *f*)—viz. lower values are better. Performance, measured by average and standard deviation across runs, is categorized into All, Old, and New classes.

(*a*) the number of per-class biflows stored during the rehearsal process, (*b*) the number of packets collected for each biflow, and (*c*) the input set extracted from biflows and used for the incremental model.

Through this analysis, our goal is to determine an optimal configuration $\langle \#mem, \#pkts, inputs \rangle$, where *#mem* indicates the number of per-class biflows, *#pkts* the number of per-biflow packets, and *inputs* the (sub)set of per-packet features leveraged—*base* stands for `[PL, IAT, DIR, WIN]` while *expanded* for `[PL, IAT, DIR, WIN, TTL, FLG]`—that enhances the classification and detection performance of our NIDS.

To achieve this, we select `BiC` since it is the top-performer both in classification and in detection tasks and systematically compare various configurations against the one analyzed in the previous sections (i.e. $\langle 25, 10, base \rangle$) across all datasets. In the sensitivity analysis, rather than conducting a grid search (i.e. Cartesian product of all combinations), we employ a sub-optimal solution by exploring a narrowed scope of solutions. This involves optimizing each parameter individually while maintaining the others at their base values (i.e. $\#mem = 25$, $\#pkts = 10$, and $inputs = base$). Best values are mainly driven by the F1 in the AC task, as there is a negligible loss w.r.t. PAUC-optimal configuration.

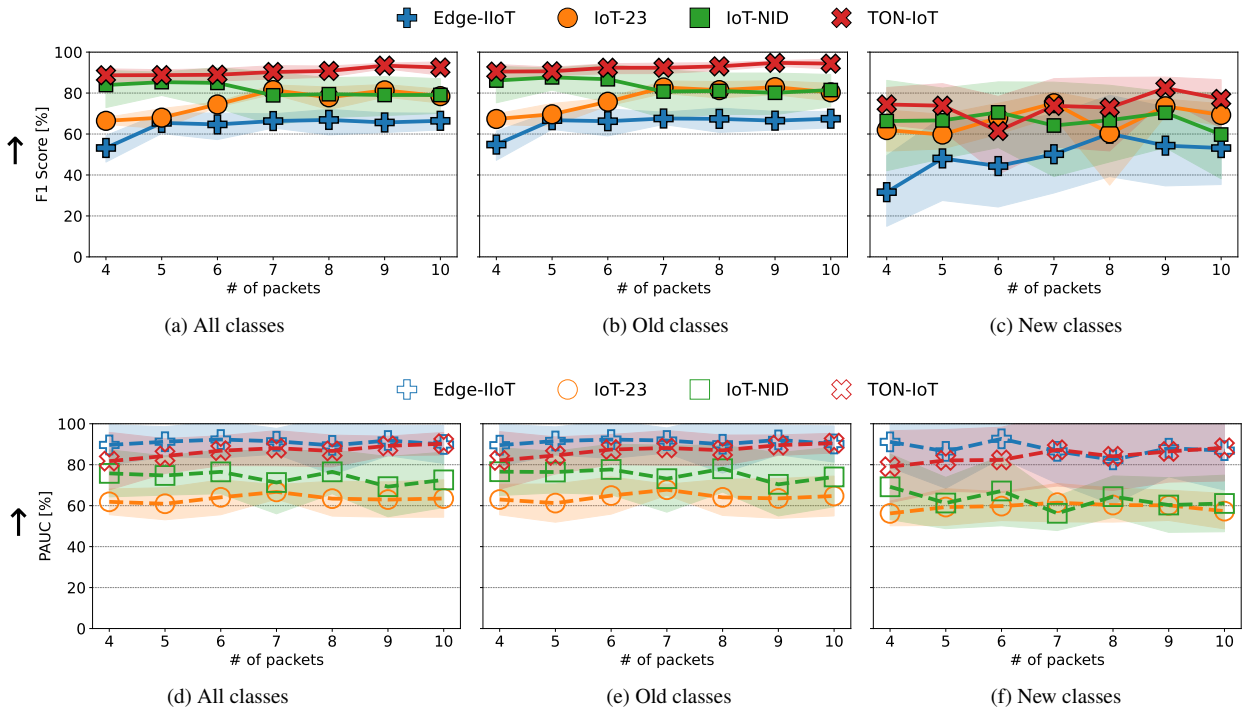
Memory Size. Herein, we examine the performance variations w.r.t. the number of per-class biflows stored in memory and subsequently represented during incremental training (viz. *#mem*). This analysis is depicted in Fig. 4 for AC and bMD tasks, both broken down by class sets—All, Old, and New.

As the memory size increases, there is a general trend of decreasing F1 DropAll across all datasets. Specifically, `IoT-23` and `Edge-IIoT` exhibit continuous improvement with increasing memory, reaching $\approx 13\%$ and $\approx 5\%$ with *#mem*=200, respectively. On the other hand, the performance of `IoT-NID` improves up to *#mem*=100, reaching $\approx 4\%$ F1 DropAll, then it experiences a slight decrease. In contrast, `TON-IoT` experiences a slight deterioration after reaching 50 biflows per class, where $\approx 4\%$ F1 DropAll is exhibited.

A similar pattern is shown for C^{all} for `IoT-23`, `Edge-IIoT`, and `IoT-NID`. Conversely, `TON-IoT` improves until *#mem*=100, obtaining a $\approx 2\%$ F1 DropOld.

However, a distinct pattern emerges for C^{new}. Notably, `IoT-23` and `Edge-IIoT` exhibit continuous enhancement in F1 DropNew as memory increases, eventually achieving optimal values of $\approx 8\%$ and $\approx 10\%$, respectively. In the case of `IoT-NID`, performance improves up to 100 biflows per class, reaching $\approx 18\%$, before experiencing a decline when utilizing $\#mem = 200$. Similarly, `TON-IoT` shows improvement with 50 biflows per class, resulting in a drop of $\approx 17\%$, but then deteriorates until reaching a drop of $\approx 48\%$ with $\#mem = 200$. These phenomena may arise from an increase in old attacks biflows, causing the model to shift its bias from new to old attacks.

As for detection performance, C^{all} and C^{old} exhibit the same pattern. Notably, `IoT-23` and `Edge-IIoT` have a decreasing trend until *#mem*=200, reaching $\approx 8\%$ and $\approx 4\%$ PAUC DropAll, respectively. For `IoT-NID`, performance has a peak with 50 per-class biflows, obtaining $\approx 9\%$ PAUC DropAll, then a decrease, and the best value is obtained with *#mem*=200 ($\approx 6\%$ PAUC DropAll). Then, `TON-IoT` continues improving until 100 per-class biflows with $\approx 2\%$ drop on C^{all} and no drop



(a) All classes (b) Old classes (c) New classes

(d) All classes (e) Old classes (f) New classes

Figure 5: Analysis of performance in terms of F1 (a, b, c) and PAUC (d, e, f) by varying the number of packets considered per each biflow—viz. higher values are better. Performance is expressed as the average and standard deviation across multiple runs and is broken down into All, Old, and New classes.

on C^{old} then performance remains stable with #*mem*=200.

Concerning C^{new}, for `Edge-IIoT` with only 50 biflows per-class we obtain a ≈ 11% PAUC Drop that remains stable for higher #*mem* values. For `IoT-23`, we obtain the best detection performance with 200 per-class biflows, exhibiting a ≈ 13% drop. For `IoT-NID`, the best performance is reached with 200 with ≈ 6% drop. Conversely, for `TON-IoT` detection performance remains quite stable, reaching the best value with #*mem*=100 (≈5% PAUC DropNew).

Take-home: From these findings, we selected #*mem*=200 for `Edge-IIoT` and `IoT-23`, #*mem*=100 for `IoT-NID`, and #*mem*=50 for `TON-IoT`, based on the best F1. It is worth noting that, while `Edge-IIoT` and `IoT-23` show the best F1 and PAUC with the same memory size, the best F1 for `TON-IoT` and `IoT-NID` results in only a 1% and 6% loss of PAUC, respectively.

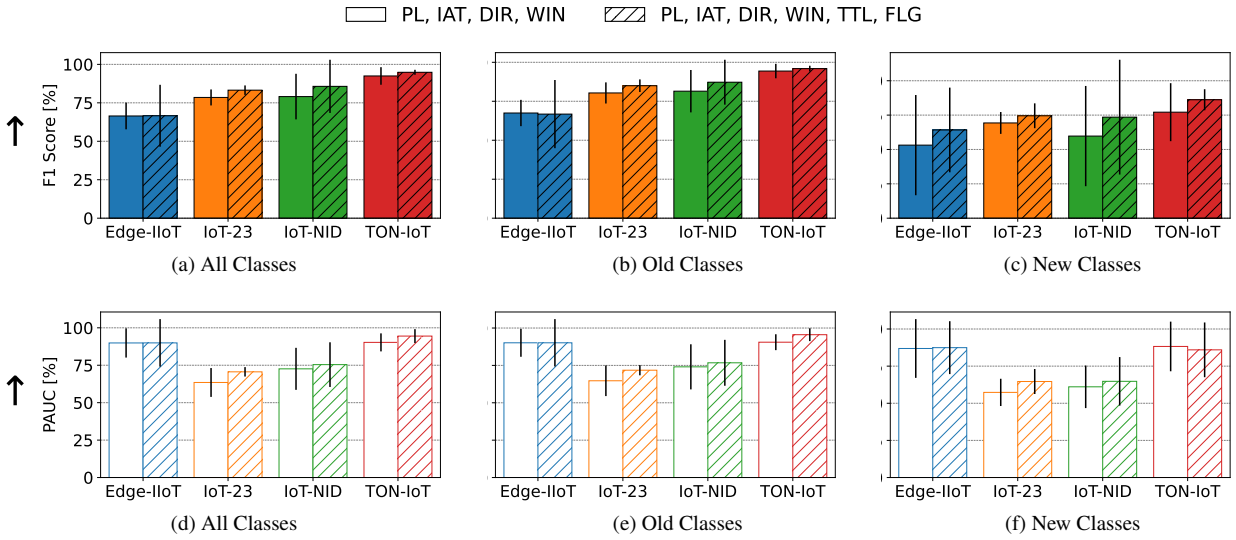
Number of Packets. Next, we examine how the performance of the incremental model is affected by varying the number of packets considered for each biflow. In our previous analysis, we fixed the number of packets for each biflow at 10, based on the previous results of the TC field. Differently, here we conduct a sensitivity analysis on the latter parameter (i.e. #*pkts*) to determine whether we can enhance the incremental classifier's earliness without compromising (and possibly even improving) overall performance. The results of this analysis are shown in Fig. 5 for AC and bMD tasks, broken down by sets of classes, namely All, Old, and New.

Specifically, the best *classification* performance on `IoT-23` is reached for C^{all}, C^{old}, and C^{new} with 7 packets, obtaining ≈ 81%, ≈ 83%, and ≈ 75%, respectively. For `Edge-IIoT`, the highest F1 is reached with 8 packets, namely ≈ 67% for C^{all} and C^{old}, and ≈ 60% for C^{new}. As for `IoT-NID`, leveraging 5 packets, we achieved ≈85%, ≈88%, and ≈67% F1 on all, old, and new classes, respectively. For new classes, the best performance (≈71%) was obtained with 6 packets. Lastly, leveraging 9 packets for `TON-IoT`, we achieved ≈93%, ≈95%, and ≈82% F1 on C^{all}, C^{old}, and C^{new}, respectively.

As for *detection* performance, we reach highest PAUC (≈ 66% on C^{all}) for `IoT-23` with #*pkts*=7. For `Edge-IIoT` and `IoT-NID`, we obtain best detection performance with #*pkts*=6, reaching a ≈ 92% and ≈ 77% PAUC All, respectively. Lastly, for `TON-IoT` best detection performance is reached with 10 packets with a ≈90% PAUC for all classes.

Although for C^{all} and C^{old} the standard deviations are contained, they are notably higher for C^{new}. This variance may be attributed to the different impacts of specific new attack classes when incrementally integrated into the model. Such effects are consistent across all utilized packet numbers and are prominently illustrated in Figs. 5c and 5f. This pattern highlights the non-trivial nature of the observed effects, underscoring the complexities involved in model adaptation to new classes. Finally, on the one hand, the non-monotonic behavior of the PAUC indicates that varying the number of packets does not affect the binary detection between benign and specific attack classes. On the other hand, tuning the number of packets parameter enables an improved separation among attack classes—thus improving the F1 for `Edge-IIoT` and `IoT-23`. Therefore, we select as the best configuration the one obtaining the higher F1 (in average).

Take-home: Based on these outcomes, we selected #*pkts*=7



(a) All Classes (b) Old Classes (c) New Classes

(d) All Classes (e) Old Classes (f) New Classes

Figure 6: Analysis of features set used as input of the incremental DL model in terms of F1 (a, b, c) and PAUC (d, e, f)—viz. higher values are better. Performance is expressed as the average and standard deviation across multiple runs and is broken down into All, Old, and New classes.

for `IoT-23`, #*pkts*=8 for `Edge-IIoT`, #*pkts*=5 for `IoT-NID`, and #*pkts*=9 for `TON-IoT`. It should be noted that, although the best PAUC and F1 values for `IoT-23` are achieved with 7 packets, this alignment is not mirrored in the other datasets. Specifically, for `IoT-NID` and `Edge-IIoT`, the highest PAUC values are attained with #*pkts*=6, while for `TON-IoT`, with #*pkts*=10, which results in a drop of F1 of 2%, 3%, and 1%, respectively.

Expanded Set of Inputs. Last, we investigate the impact of leveraging an expanded set of input (i.e. `[PL, IAT, DIR, WIN, TTL, FLG]` w.r.t. the *base* set (i.e. `[PL, IAT, DIR, WIN]`). For this analysis, we use the same set of inputs for the classifier, both in the base and incremental steps. In this section, we explore how leveraging different sets of features impacts the tasks of classification and detection, respectively. It is important to emphasize that these two tasks may be differently influenced by modifications in the input features. The results of this analysis are shown in Fig. 6 for the AC task and bMD tasks, broken down by sets of classes.

For classification performance, we observe an increase across all datasets, but `Edge-IIoT`, when using the extended set of features. Specifically, for `IoT-23` we reach ≈ 78%, for `IoT-NID` ≈ 86%, and for `TON-IoT` ≈ 95% on C^{all}. Alongside this enhancement, performance also improves on both C^{old} and C^{new}. For instance, with the extended feature set, performance on new classes improves notably on `IoT-NID`, rising from ≈ 60% to ≈ 74%. On the other hand, for `Edge-IIoT`, we obtain similar performance (≈ 66% on C^{all}) with the extended set of features but with high variability (i.e. high *standard deviation*).

In terms of detection performance, significant improvements are observed across all datasets and subsets of classes except for `Edge-IIoT` where PAUC remains stable around ≈ 90%. Specifically, we achieve ≈ 71% accuracy on `IoT-23`, ≈ 75% on `IoT-NID`, and ≈94% on `TON-IoT` for the C^{all} subset.

Take-home: Therefore, we leverage *inputs=base* for `Edge-IIoT` and *inputs=expanded* for all other datasets since we found consistent results between F1 and PAUC.

5.4. Wrap-up Discussion

In this section, we evaluate the performance enhancements attained through the selection of the best parameter configuration for memory size (#*mem*), packet number (#*pkts*), and feature set (*features*). The outcomes of this evaluation are detailed in Tab. 7. Specifically, `IoT-23` demonstrates improved performance with 7 packets per biflow, an expanded feature set, and 200 samples per class. Regarding classification of attacks, F1 reaches ≈ 87% (+9% compared to the base configuration) for C^{all}, ≈ 88% (+8%) for C^{old}, and ≈ 83% (+13%) for C^{new}. Similarly, detection performance sees an increase of ≈ 10% for C^{all} and ≈ 9% for C^{old}, reaching ≈ 73% and ≈ 74%, respectively. For C^{new}, the new configuration reaches ≈ 70% (+13% compared to the base configuration).

Furthermore, with 9 packets per biflow, an expanded feature set, and 50 samples per class, `TON-IoT` experiences a performance enhancement of ≈ 3% for C^{all} and C^{old} (achieving ≈ 95% and ≈ 96%, respectively), and ≈ 9% for C^{new} (≈ 86% F1). In terms of detection, there is an improvement of approximately ≈ 7% for C^{all} and C^{old}, and ≈ 1% for C^{new}.

Subsequently, with #*pkts*=5, *features=expanded*, and #*mem*=100, `IoT-NID` exhibits a performance enhancement of ≈ 18% compared to the base configuration, achieving ≈ 97% across all attack classes. Likewise, for Old (resp. New) classes, performance increases by ≈ 16% (resp. ≈ 35%), reaching ≈ 98% (resp. ≈ 94%) F1. Regarding PAUC, this configuration boosts detection performance by ≈ 15% for C^{all}, ≈ 14% for C^{old}, and ≈ 21% for C^{new}.

Finally, `Edge-IIoT` demonstrates a performance increase of 6% for C^{all} and ≈ 5% for C^{old}, resulting in a ≈ 72% F1 value, and obtains ≈ 68% for C^{new} (+15% w.r.t. base configuration), using 8 packets per biflow, the base feature set, and 200 samples per

Table 7: Table illustrating the performance and Average Waiting Time (AWT) of BiC in both its initial configuration and its optimal configuration selected in previous *sensitivity analyses*. For each dataset, we indicate the difference in F1 Score, PAUC, and AWT of the best configuration (highlighted with gray colored cells) over the base one: higher values are the better. AWT refers to the average time needed to wait for classifying the biflow.

Dataset	Config.			F1 Score [%]			PAUC [%]			AWT [ms]
	#mem	*#pkts*	*inputs*	**All**	**Old**	**New**	**All**	**Old**	**New**	
IoT-23	25	10	*base*	78.49±4.71	80.33±6.29	69.29± 7.40	63.49±9.13	64.72±9.74	57.32±8.66	984.39
	200	7	*expanded*	87.18±1.76	88.11±1.49	82.53±12.01	73.10±8.09	73.66±9.49	70.29±9.47	977.20
				+8.69	**+7.78**	**+13.24**	**+9.61**	**+8.94**	**+12.97**	**−7.19**
TON-IoT	25	10	*base*	92.47±5.21	94.38±4.15	77.13±20.54	90.27±5.51	90.52±4.81	88.25±16.20	3884.40
	50	9	*expanded*	95.30±1.46	96.47±1.18	85.89± 7.76	96.79±2.64	97.75±2.04	89.03±18.58	1615.40
				+2.83	**+3.42**	**+8.76**	**+6.52**	**+7.23**	**+0.78**	**−2269.00**
IoT-NID	25	10	*base*	79.06±14.32	81.47±13.11	59.74±35.92	72.60±13.52	74.04±14.58	61.08±13.81	2546.26
	100	5	*expanded*	97.42± 1.05	97.80± 1.20	94.36± 8.60	87.10± 4.43	87.67± 4.36	82.25±12.43	2318.89
				+18.36	**+16.33**	**+34.62**	**+14.50**	**+13.63**	**+21.17**	**−227.36**
Edge-IIoT	25	10	*base*	66.45±8.18	67.47±7.95	53.16±35.91	89.89±9.26	90.12±8.84	86.88±19.27	3692.53
	200	8	*base*	72.05±6.31	72.33±6.14	68.36±31.62	93.99±5.61	94.40±5.46	88.47±15.83	2224.44
				+5.60	**+4.86**	**+15.20**	**+4.10**	**+4.28**	**+1.59**	**−1468.08**

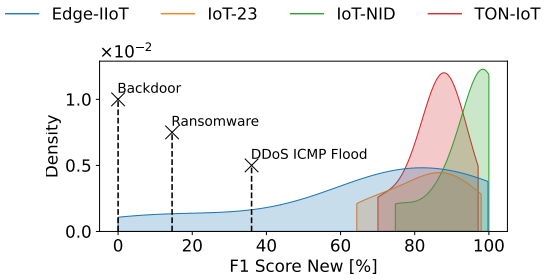


Figure 7: F1 Score New distribution for all datasets.

class. Moreover, detection performance sees an enhancement of ≈4% for all and old classes, with a 2% improvement observed for the new attack class.

In terms of Average Waiting Time (AWT), an improvement is experienced for all the datasets. Notably, TON-IoT and Edge-IIoT exhibit the maximum improvement of ≈ 2269 ms and ≈ 1468 ms, respectively, while IoT-NID and IoT-23 save only ≈227 ms and ≈ 7 ms. The difference could stem from the nature of attacks exhibiting longer inter-arrival times within the initial N packets for IoT-23 and IoT-NID. Moreover, IoT-23 notably features fewer occurrences of slow-rate attacks compared to other datasets.

6. Real-World Challenges for Intrusion Detection Systems

In this section, we delve into several real-world challenges for NIDS. These include the need for transparent decision-making processes to promote widespread adoption of such models (Sec. 6.1), the robustness against input perturbations and changes in network conditions (Sec. 6.2), and the capability to effectively include new attacks even with limited available data (Sec. 6.3). To perform these analyses, we leverage the best configuration obtained for each dataset (cf. Sec. 5.4).

6.1. Explainability

Aim of the analysis: In this section, we comprehensively examine the performance of 0-day attacks (C^{new}) across all datasets, analyzing each attack addition to identify factors behind the observed performance challenges. Specifically, we focus on C^{new} due to the low F1 values and significant variability, suggesting the presence of *critical* 0-*day attack addition*.
As such, Fig. 7 illustrates the distribution of F1 for new attacks. Specifically, Edge-IIoT demonstrates the most skewed distribution, with 3 specific attacks achieving < 40% F1—namely *Backdoor* (0%), *Ransomware* (≈ 15%), *DDoS ICMP Flood* (≈36%). While other datasets display more concentrated distributions around the average, they also feature attacks with lower F1 values—i.e. *C&C* for IoT-23 (≈ 64%), *OS/Version Scanning* for IoT-NID (≈75%), and *Injection* for TON-IoT (≈70%).

To delve deeper into the analysis of these 0-day attack additions, we present confusion matrices in Fig. 8. Notably, in Fig. 8a, we visualize the predictions for the *Backdoor* attack for BiC, demonstrating a total confusion with the *Uploading* attack by the incremental model. However, a similar situation is experienced also in Scratch predictions (Fig. 8b) with only a ≈10% of *Backdoor* biflows correctly classified. Indeed, both attacks involve outbound data transfers, typically to remote servers, indicative of command and control activity, thereby sharing network traffic patterns.

In the case of IoT-23 (see Figs. 8c and 8d), confusion between the *C&C* and the *C&C-Heartbeat* attack is evident for both BiC and Scratch, aligning with expectations due to their shared command and control behavior, reflected in the predictions of the incremental model. This confusion leads to a model imbalance toward *C&C* in the incremental model (as it is the last-added attack) and toward *C&C-Heartbeat* in Scratch (as it is the most populous between the two attack classes).

Regarding IoT-NID (see Figs. 8e and 8f), it is noteworthy that some *ARP Spoofing* biflows are erroneously classified as *OS/Version Scanning* by BiC while the *training-from-scratch*

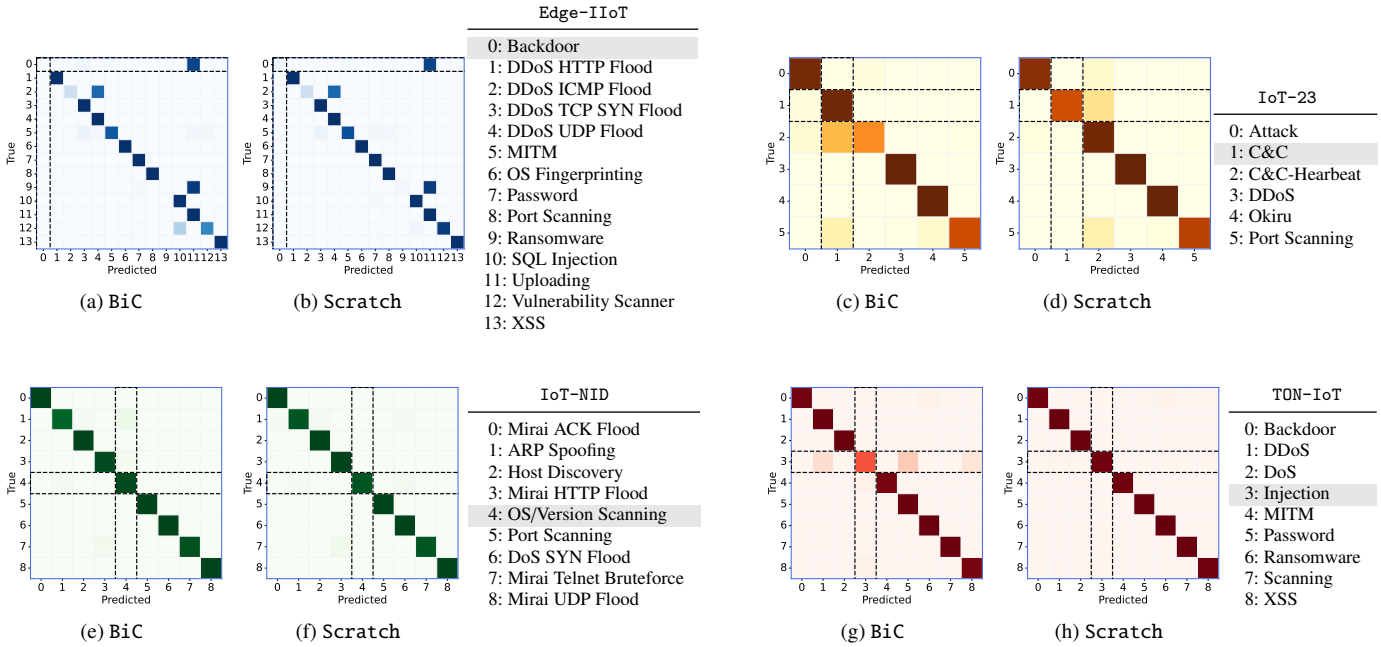


Figure 8: Fine-grained analysis focusing on the specific addition of the last class for BiC and Scratch. The last class under consideration is emphasized with a gray background in the relative table and with a dashed line in the relative figure.

better separates these two attacks. Incremental learning performance could be affected by the shared use of ARP packets by both of them, potentially leading to overlapping patterns in traffic analysis despite serving different purposes.

Finally, within the context of TON-IoT (see Fig. 8g), it is observed that the *Injection* attack shares behavioral traits with the *DDoS*, *Password*, and *XSS* ones. Conversely, Scratch (Fig. 8h) correctly classifies the *Injection* attack. BiC confusion can be due to the disruptive nature of such attacks, causing unusual traffic spikes or abnormal request/response behaviors, which may be mistakenly identified as other types of attacks.

In the following, we focus on the Edge-IIoT case, which exhibits the most critical performance scenarios for C^{new}. Notably, we employ XAI techniques to gain insights into the decision-making process. The three previously highlighted attacks also demonstrate low F1 in the Scratch model, with *Backdooor* achieving $\approx$ 10%, *DDoS ICMP Flood* $\approx$ 36%, and *Ransomware* $\approx$ 4%. Since the last two exhibit comparable or even better performance in the incremental and Scratch model, we focus our analysis on the *Backdoor* attack.

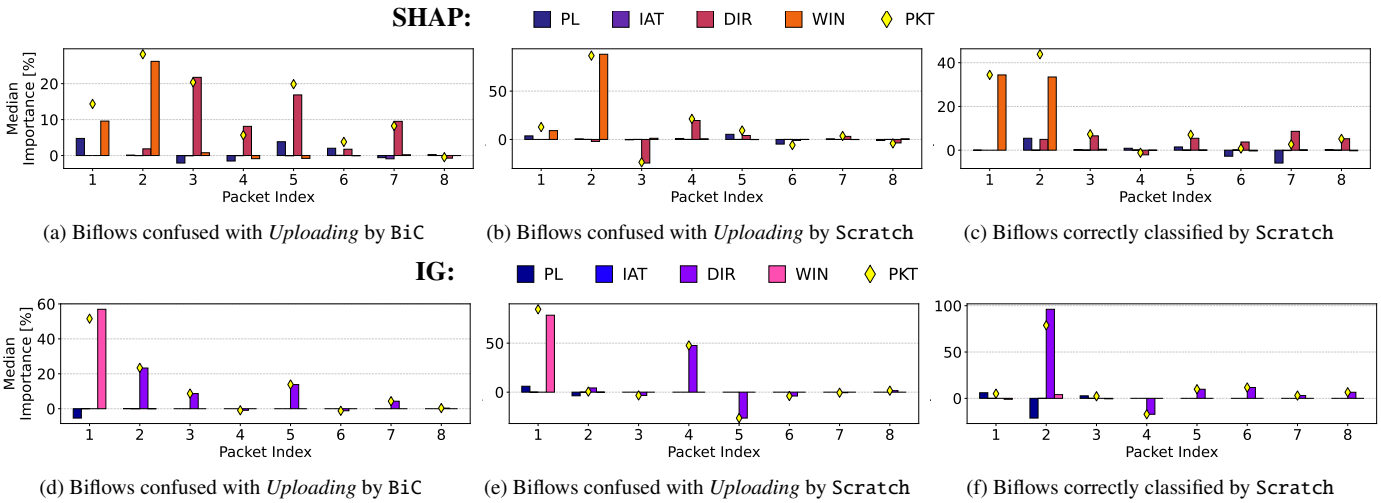
Accordingly, we investigate the input importance with SHAP and IG for *Backdoor* biflows in different cases to understand the decision-making process (Fig. 9): (*a*) when they are confused with *Uploading* attack by BiC and Scratch, (*b*) when they are correctly classified by Scratch.

In the former case, according to SHAP, the most influential input fields for BiC are the WIN of 1st and 2nd packet and DIR from the 3nd packet to the 7th one (Fig. 9a). On the other hand, although IG also identifies WIN and DIR as the most important input fields, in this case, it only considers the first WIN and the directions of the 2nd, 3rd, 5th, and 7th packets (Fig. 9d). For Scratch, SHAP identifies the most important fields in the WIN of the first two packets and in the DIR of 3rd (*negative influence*) and 4th (*positive influence*) packets (Fig. 9b). Instead, IG poses more importance on the first WIN and 4th (positive) and 5th (negative) packets (Fig. 9e). It is evident that while both BiC and Scratch misclassify *Backdoor* biflows with *Uploading* attacks, they make decisions relying on different sets of input fields. However, they share similarities in only the 2nd WIN and 4th DIR for SHAP, and the 1st WIN for IG.

In the latter case, the importance is assigned by SHAP (see Fig. 9c) mostly to the first two WIN values and then spread across the PL and the DIR of the last six packets. Conversely, IG recognizes as the most influential input field the WIN (mostly of the 2nd packets) and the PL of the first two packets to a lesser extent (Fig. 9f).

To quantify the coherence of explanations—i.e. the concordance between the two input-attribution techniques—we compute the $Top5Ratio$ and $Avg.Rel.$ (cf. Sec.4.1) of the explanations illustrated in Fig. 9. Notably, for *Backdoor* biflows misclassified by BiC, the $Top5Ratio$ is 80%, underscoring agreement between SHAP and IG regarding the most crucial fields for decision-making. Similarly, the two explanations exhibit an $Avg.Rel.$ of $\approx 5 \times 10^{-2}$, confirming only a slight difference between the two methods. For Scratch, in the case of misclassification, the the $Top5Ratio$ is 6% while $Avg.Rel.$ is $\approx 8 \times 10^{-2}$. Conversely, for the correct classification, the $Top5Ratio$ is 40% while $Avg.Rel.$ is $\approx 9 \times 10^{-2}$.

Take-home: In conclusion, **poor performance for specific 0-day attacks can be influenced by their *similarity* to previously-encountered attacks** and further worsened by the incremental paradigm. Furthermore, **different input attribution techniques consistently produce coherent results for the incremental model** while explanations have a weaker struc-



(a) Biflows confused with *Uploading* by `BiC` (b) Biflows confused with *Uploading* by `Scratch` (c) Biflows correctly classified by `Scratch`

(d) Biflows confused with *Uploading* by `BiC` (e) Biflows confused with *Uploading* by `Scratch` (f) Biflows correctly classified by `Scratch`

Figure 9: Explanations for *Backdoor* biflows confused as *Uploading* attack in `Edge-IIoT` by `BiC` (*a*, *d*) and `Scratch` (*b*, *e*) or correctly classified by `Scratch` (*c*, *f*) with SHAP (*a*, *b*, *c*) and IG (*d*, *e*, *f*).

tural similarity and greater variability with respect to the extreme values for *trained-from-scratch* model.

Table 8: Classification and detection performance on C^{all} for `BiC` and `Scratch` under varied perturbations of input data. First row shows the absolute performance without perturbation for each dataset (**in boldface**). The next rows indicate the deviation from this baseline when perturbations are applied to the input—lower values are the better (denoted by the **"Drop →"** column). **Orange** cells indicate perturbation for which `BiC` and `Scratch` differs more than 5% while **red** text highlight perturbations with ≥20% F1 Drop.

		F1 Score [%]		PAUC [%]		
	Perturbation	`BiC`	`Scratch`	`BiC`	`Scratch`	
`IoT-23`	**None**	**87.18**	**90.31**	**73.10**	**82.72**	
	Timeout	2.90	2.93	10.05	19.05	**Drop ↓**
	Jittering	22.74	18.16	17.91	21.96	
	Flipping	55.82	52.52	21.74	27.19	
	Slicing	78.38	80.43	23.04	32.77	
`TON-IoT`	**None**	**95.30**	**98.17**	**96.79**	**99.06**	
	Timeout	4.37	4.12	1.28	1.99	**Drop ↓**
	Jittering	13.26	15.92	4.21	4.99	
	Flipping	64.42	72.93	29.38	37.65	
	Slicing	82.33	80.64	36.62	37.34	
`IoT-NID`	**None**	**97.42**	**98.61**	**87.10**	**94.16**	
	Timeout	3.69	4.58	10.41	0.53	**Drop ↓**
	Jittering	0.05	0	0.21	0.26	
	Flipping	77.22	68.98	32.98	35.52	
	Slicing	81.93	79.74	34.13	40.88	
`Edge-IIoT`	**None**	**72.05**	**78.24**	**93.99**	**98.78**	
	Timeout	10.30	14.68	23.57	25.59	**Drop ↓**
	Jittering	25.35	40.11	30.10	26.66	
	Flipping	54.70	65.74	21.06	20.26	
	Slicing	65.49	71.65	26.98	30.48	

6.2. Robustness

Aim of the analysis: In this section, we delve into the real-world implications of non-idealities associated with the input of a DL model. Notably, we investigate the impact of input perturbations on classification and detection performance.

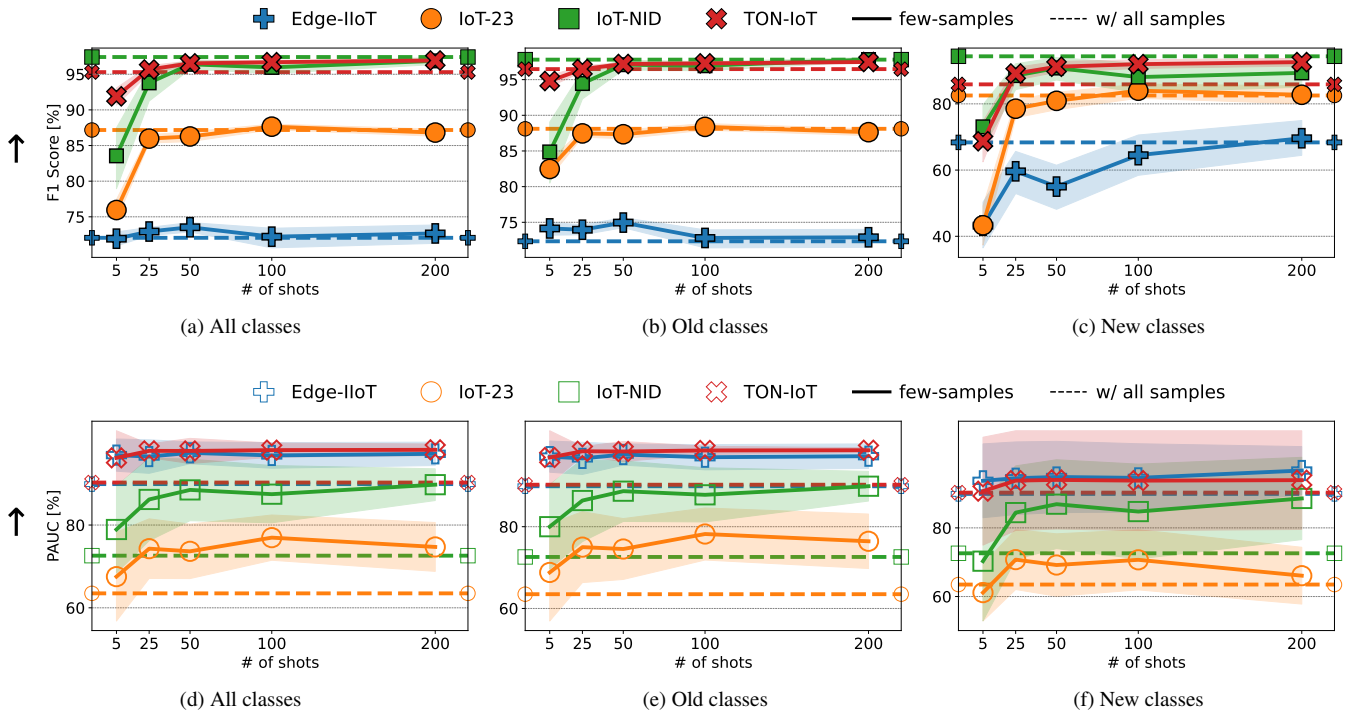
We report corresponding performance under the different input perturbations for `BiC` and `Scratch` in Tab. 8.

Specifically, the *slicing* perturbation leads to poor classification (< 18% F1) and detection performance (< 69% PAUC) for both `BiC` and `Scratch` on the considered datasets. This implies that solely analyzing the initial phase of communication (viz. the first part of a biflow) does not suffice for the NIDS to detect and classify an attack effectively.

As for the *flipping* manipulation, this disruption significantly impacts classification performance, with all datasets experiencing a < 50% F1, particularly evident with `Edge-IIoT` with the incremental model (≈ 17%). Furthermore, `Scratch` achieves similar performance to `BiC`, even underperforming it on `Edge-IIoT` and `TON-IoT`. Detection performance also suffers, falling below 60% PAUC for `IoT-23` and `IoT-NID`, in both `Scratch` and `BiC`. Notably, in `TON-IoT`, `BiC` achieves a ≈ 68% PAUC, surpassing even `Scratch` (≈ 61% PAUC), while on `Edge-IIoT`, `BiC` achieves a ≈ 73% PAUC while `Scratch` ≈ 79%. These results suggest that packet direction plays a significant role in attack classification for both incremental and non-incremental models, while for detection, this holds across all datasets except `Edge-IIoT`, where `Scratch` exhibits a smaller loss w.r.t. the unperturbed scenario.

On the other hand, for the *timeout* perturbation, the classification performance is slightly impacted for `IoT-23` and `IoT-NID` with −3% F1, and `TON-IoT` with −4% F1 w.r.t. *no perturbation* case. Instead, for `Edge-IIoT` we experience a −10% F1. Concerning detection performance, for `TON-IoT` it remains quite similar w.r.t. the unperturbed scenario while all the other datasets experience a significant decay −10% PAUC for `IoT-23` and `IoT-NID`, and −24% PAUC for `Edge-IIoT`.

Regarding the *jittering* perturbation, for `IoT-NID` the classification performance is only slightly impacted suggesting that `BiC` relies minimally on IAT for the attack classification on this dataset. This is confirmed also by stable detection performance.



(a) All classes (b) Old classes (c) New classes

(d) All classes (e) Old classes (f) New classes

Figure 10: Analysis in a few-shot scenario of best configurations for leveraged datasets in terms of F1 (a, b, c) and PAUC (d, e, f).

For the other dataset, the inter-arrival time between consecutive packets holds much greater significance in distinguishing benign traffic from malicious traffic (with more than 10% decay w.r.t. unperturbed scenario). This also holds for detection on `IoT-23` and `Edge-IIoT` where a $\approx$ 18% and a $\approx$ 30% PAUC degradation is experienced. Conversely, for `TON-IoT` the detection performance remains quite stable suggesting that the IAT is not significant in distinguishing benign and malicious traffic.
Take-home: In conclusion, both incremental and non-incremental models are **not robust to perturbation affecting the packet direction and the number of visible packets**; however, they show **good performance against** perturbations that simulate **packet loss or vary the packet inter-arrival time**.

6.3. Including few-shot 0-day attacks

Aim of the analysis: A common challenge for NIDS lies in gathering attack samples necessary to train ML and DL approaches, which demand substantial amounts of data to achieve efficacy. In this section, we explore the efficacy of the optimal `BiC` configuration in an FSCIL scenario—where the incremental phase operates with a limited number of samples (referred to as *shots*) for a newly detected 0-day attack.
Precisely, we assess its performance across varying shot numbers within the set $K \in \{5, 25, 50, 100, 200\}$. Furthermore, for this analysis, we made slight adjustments to `BiC` concerning the composition percentage of samples in $\mathcal{D}^{corr}$ to ensure the presence of training data. Specifically, when $K < N_{old}$, we construct the correction dataset using $\lceil 10\% \cdot K \rceil$ samples. Conversely, in cases where $K \geq N_{old}$, we employ the original `BiC` formulation.

The findings from this analysis are depicted in Fig. 10, showcasing the performance achieved by the optimal configuration of `BiC`. Additionally, the results include the performance attained when utilizing all available samples for the new attacks, denoted as "All Samples" in the figure legend.

With only 5 shots, the performance exhibits a notable disparity w.r.t. the "All Samples" scenario. This difference is mainly evident for new classes where we reach a $\approx$ 43% F1 on `Edge-IIoT` and `IoT-23`, $\approx$73% on `IoT-NID`, and $\approx$68% on `TON-IoT`.

However, as the number of shots increases, the performance of `BiC` improves consistently. For `IoT-23`, the performance surpasses that of the CIL scenario once 100 shots are employed for all subsets of classes ($\approx$ 88% for C^{all} and C^{old}, $\approx$ 84% per C^{new}), remaining quite stable at 200 shots.

Similarly, in the `Edge-IIoT` domain, performance at 25 shots is similar to that of the CIL scenario. Specifically, the performance reaches $\approx$73% for C^{all}, $\approx$74% for C^{old}, and $\approx$60% for C^{new}. This stability persists up to 200 shots, where the performance reaches $\approx$73% for C^{all} and C^{old}, and $\approx$70% for C^{new}.

In the case of `IoT-NID`, the performance of FSCIL scenarios is lower than the CIL one across all the shots' numbers. In particular, we obtain $\approx$97% on C^{all}, $\approx$98% on C^{old}, and $\approx$89% on C^{new} at 200 shots.

Lastly, on `TON-IoT`, performance begins to rise from 25 shots onwards, matching and eventually surpassing the results obtained without any constraints on the samples of the new class ($\approx$ 96% for C^{all} and C^{old}, $\approx$ 89% for C^{new}). Notably, performance slightly continues to improve for the C^{old} and C^{new} subsets, reaching $\approx$ 97% for C^{all} and C^{old}, and $\approx$ 93% for C^{new} at 200 shots.

As for detection, the performance exhibits a unimodal trend, peaking at a specific point for each dataset (i.e., 50 shots for

Edge-IIoT, 100 for IoT-23, 200 for IoT-NID, and 25 for TON-IoT). Generally, the performances achieved under the *few-samples constraint* surpass those of the CIL model. This is probably due to the reduced catastrophic forgetting experienced by the model.

Take-home: These findings highlight **the significant impact of low shot numbers (i.e. 5 and 15) on classification performance**, with noticeable improvements observed as the shot number increases (from 50 onwards), effectively mitigating imbalance issues and enhancing overall performance. Conversely, **detection performance shows an unimodal trend and remains effective under few-sample constraints**, outperforming the CIL model.

7. Conclusion

The IoT rapid growth brings enhanced connectivity and convenience but also significant cybersecurity challenges. With malicious actors targeting IoT vulnerabilities, the need for resilient security measures is crucial. CIL offers a dynamic strategy empowering ML and DL models to evolve and adapt to emerging attack patterns while detecting known threats effectively. This approach holds promise for strengthening NIDS resilience in IoT networks.

In this study, we tackled research questions (**Q1**-**Q5**) related to NIDS challenges in real-world scenarios, providing answers to each (**A1**-**A5**). We evaluated the effectiveness of cutting-edge CIL approaches in *detecting & classifying* 0*-day attacks*. Notably, (**A1**) BiC emerges as the top performer in detecting (viz. binary misuse detection) and classifying (viz. attack classification) in an incremental setup, exhibiting at most $\approx 18\%$ F1 drop and $\approx 19\%$ PAUC drop w.r.t. a *trained-from-scratch* model but saving up to $\approx 80\%$ of its training time. Additionally, (**A2**) adjusting key hyper-parameters leads to improved performance for adaptive NIDS, obtaining up to $\approx 18\%$ improvement in F1 and $\approx 15\%$ in PAUC, fostering the adoption of this incremental solution in real-world environments.

Then, we dealt with practical hurdles for NIDS, such as explainability, robustness, and few-sample availability.

Specifically, (**A3**) we investigated the subpar performance of certain 0-day attacks with XAI techniques. We identified the cause of this performance issue mainly in the similarities between the 0-day attacks and already encountered ones but also further degradation is due to the incremental paradigm. Moreover, we validated the explanations obtained from different feature attribution techniques (i.e. SHAP and IG). We confirmed the coherence in the outcomes of these two methods for the incremental model ($Top5Ratio = 80\%$ and $Avg.Rel. = 5 \times 10^{-2}$). Conversely, for *trained-from-scratch* model, explanations have a weaker structural similarity ($Avg.Rel. \geq 8 \times 10^{-2}$), and higher variability regarding the extreme values ($Top5Ratio \geq 40\%$).

As for robustness, (**A4**) both incremental and non-incremental models maintain satisfactory performance to input perturbation affecting inter-arrival time or simulating packet loss. For instance, they experience at most $\approx 25\%$ (resp. ≈ 40) F1 drop and $\approx$ 30% (resp. $\approx$ 27%) PAUC drop w.r.t. BiC (resp. Scratch) *without perturbation*. Conversely, both models struggle with perturbations affecting packet direction and limiting packet visibility—obtaining at most F1 performance $< 10\%$ and $\approx 37\%$ PAUC drop for both BiC and Scratch. Furthermore, across all perturbations, BiC and Scratch exhibit comparable drops in performance, except for Edge-IIoT in the classification task and IoT-23 in the detection task, where their performance differs by more than 5%.

Lastly, to address the scarcity of samples in real-world scenarios, we tested our solution with constraints on number of samples available. We found that (**A5**) classification performance is affected with very few samples (i.e. 5 and 15 for new attacks) while for high numbers they reach or even surpass CIL (viz. *all samples*) scenario. On the other hand, detection performance is not affected by the fewness of samples exhibiting stronger performance w.r.t. the *all samples* scenario.

As *future directions*, we will delve into further meaningful evaluation scenarios within the specified domain, also evaluating the generalization capability of a NIDS when deployed in different networks. Our focus extends to exploring the application of multimodal deep learning architectures, to leverage the diversity inherent in network traffic to optimize the effectiveness of CIL approaches, or *more sophisticated CIL approaches* [44, 53]. This includes considering the payload data as DL input, particularly in scenarios where privacy concerns allow its usage [48]. Furthermore, we are keen on investigating CIL solutions that dynamically adapt to notable shifts in network traffic patterns, as highlighted in [54]. This adaptive approach aims to accommodate variations arising from changes in attacker behavior or the emergence of new threat variants, effectively mitigating *intra-class concept-drift*.

Acknowledgment

This work is partially supported by the Italian PNRR MUR "Centro Nazionale HPC, Big Data e Quantum Computing, Spoke9 - Digital Society & Smart Cities"; in part by the "xInternet" Project—funded by MUR—within the PRIN 2022 program (D.D.104-02/02/2022); and in part by the European Union under the Italian National Recovery and Resilience Plan (NRRP) of NextGenerationEU, partnership on "Telecommunications of the Future" (PE00000001 - program "RESTART"). This manuscript reflects only the authors' views and opinions and the Ministry cannot be considered responsible for them.

References

[1] Jayavardhana Gubbi, Rajkumar Buyya, Slaven Marusic, and Marimuthu Palaniswami. Internet of Things (IoT): A vision, architectural elements, and future directions. *Future Generation Computer Systems*, 29(7):1645–1660, 2013.

[2] Fadele Ayotunde Alaba, Mazliza Othman, Ibrahim Abaker Targio Hashem, and Faiz Alotaibi. Internet of Things security: A survey. *Journal of Network and Computer Applications*, 88:10–28, 2017.

[3] Ciro Guida, Alfredo Nascita, Antonio Montieri, and Antonio Pescapé. Cross-evaluation of deep learning-based network intrusion detection systems. In *IEEE 10th International Conference on Future Internet of Things and Cloud (FiCloud)*, 2023.

[4] Giampaolo Bovenzi, Giuseppe Aceto, Domenico Ciuonzo, Antonio Montieri, Valerio Persico, and Antonio Pescapé. Network anomaly detection methods in IoT environments via deep learning: A fair comparison of performance and robustness. *Computers & Security*, 128:103167, 2023.

[5] Yang Guo. A review of machine learning-based zero-day attack detection: Challenges and future directions. *Computer Communications*, 198:175–185, 2023.

[6] Idio Guarino, Giampaolo Bovenzi, Davide Di Monda, Giuseppe Aceto, Domenico Ciuonzo, and Antonio Pescapé. On the use of machine learning approaches for the early classification in network intrusion detection. In *IEEE International Symposium on Measurements & Networking (M&N)*, pages 1–6, 2022.

[7] Tingting Wang, Qiujian Lv, Bo Hu, and Degang Sun. A few-shot class-incremental learning approach for intrusion detection. In *IEEE International Conference on Computer Communications and Networks (ICCCN)*, pages 1–8, 2021.

[8] Sebastian Garcia, Agustin Parmisano, and Maria Jose Erquiaga. IoT-23: A labeled dataset with malicious and benign IoT network traffic, 2020. URL `https://www.stratosphereips.org/datasets-iot23`.

[9] Tim M Booij, Irina Chiscop, Erik Meeuwissen, Nour Moustafa, and Frank TH den Hartog. ToN_IoT: The role of heterogeneity and the need for standardization of features and attack types in IoT network intrusion datasets. *IEEE Internet Things J.*, 2021.

[10] Hyunjae Kang, Dong Hyun Ahn, Gyung Min Lee, Jeong Do Yoo, Kyung Ho Park, and Huy Kang Kim. IoT network intrusion dataset, 2019. URL `https://dx.doi.org/10.21227/q70p-q449`.

[11] Mohamed Amine Ferrag, Othmane Friha, Djallel Hamouda, Leandros Maglaras, and Helge Janicke. Edge-IIoTset: A new comprehensive realistic cyber security dataset of IoT and iIoT applications for centralized and federated learning. *IEEE Access*, 10:40281–40306, 2022.

[12] Kalle Burbeck and Simin Nadjm-Tehrani. Adaptive real-time anomaly detection with incremental clustering. *Information Security Technical Report*, 12(1):56–67, 2007.

[13] Cheng Zhong and Na Li. Incremental clustering algorithm for intrusion detection using clonal selection. In *IEEE Pacific-Asia Workshop on Computational Intelligence and Industrial Application (PACIIA)*, volume 1, pages 326–331, 2008.

[14] Wei-Yi Yu and Hahn-Ming Lee. An incremental-learning method for supervised anomaly detection by cascading service classifier and ITI decision tree methods. In *Springer Intelligence and Security Informatics: Pacific Asia Workshop (PAISI)*, pages 155–160, 2009.

[15] Yang Yi, Jiansheng Wu, and Wei Xu. Incremental SVM based on reserved set for network intrusion detection. *Expert Systems with Applications*, 38 (6):7698–7707, 2011.

[16] Binhan Xu, Shuyu Chen, Hancui Zhang, and Tianshu Wu. Incremental k-NN SVM method in intrusion detection. In *IEEE 8th International Conference on Software Engineering and Service Science (ICSESS)*, pages 712–717, 2017.

[17] Christos Constantinides, Stavros Shiaeles, Bogdan Ghita, and Nicholas Kolokotronis. A novel online incremental learning intrusion prevention system. In *IFIP 10th International Conference on New Technologies, Mobility and Security (NTMS)*, pages 1–6, 2019.

[18] Mahendra Data and Masayoshi Aritsugi. An incremental learning algorithm on imbalanced data for network intrusion detection systems. In *ACM 10th International Conference on Computer and Communications Management (ICCCM)*, pages 191–199, 2022.

[19] Hao Xu and Yongjun Wang. A continual few-shot learning method via meta-learning for intrusion detection. In *IEEE 4th International Conference on Civil Aviation Safety and Information Technology (ICCASIT)*, pages 1188–1194, 2022.

[20] Alec F Diallo and Paul Patras. Cluster and conquer: Malicious traffic classification at the edge. *IEEE Trans. Netw. Service Manag.*, 2023.

[21] Lei Du, Zhaoquan Gu, Ye Wang, Le Wang, and Yan Jia. A few-shot class-incremental learning method for network intrusion detection. *IEEE Trans. Netw. Service Manag.*, 21(2):2389–2401, 2024.

[22] Zhuoxue Song, Ziming Zhao, Fan Zhang, Gang Xiong, Guang Cheng, Xinjie Zhao, Shize Guo, and Binbin Chen. I^2RNN: an incremental and interpretable recurrent neural network for encrypted traffic classification. *IEEE Trans. Depend. Sec. Comput.*, 2023.

[23] Francesco Cerasuolo, Giampaolo Bovenzi, Christian Marescalco, Francesco Cirillo, Domenico Ciuonzo, and Antonio Pescapè. Adaptive intrusion detection systems: Class incremental learning for IoT emerging threats. In *IEEE International Conference on Big Data (BigData)*, pages 3547–3555, 2023.

[24] Giampaolo Bovenzi, Alfredo Nascita, Lixuan Yang, Alessandro Finamore, Giuseppe Aceto, Domenico Ciuonzo, Antonio Pescapé, and Dario Rossi. Benchmarking class incremental learning in deep learning traffic classification. *IEEE Trans. Netw. Service Manag.*, 21(1):51–69, 2024.

[25] Idio Guarino, Chao Wang, Alessandro Finamore, Antonio Pescapè, and Dario Rossi. Many or few samples? comparing transfer, contrastive and meta-learning in encrypted traffic classification. In *IFIP Traffic Monitoring Analysis (TMA)*, 2023.

[26] Jielun Zhang, Fuhao Li, Feng Ye, and Hongyu Wu. Autonomous unknown-application filtering and labeling for DL-based traffic classifier update. In *IEEE Conference on Computer Communications (INFOCOM)*, 2020.

[27] Zhizhong Li and Derek Hoiem. Learning without forgetting. *IEEE Trans. Pattern Anal. Mach. Intell.*, 40(12):2935–2947, 2017.

[28] J. Kirkpatrick, R. Pascanu, N. Rabinowitz, J. Veness, G. Desjardins, A. A. Rusu, K. Milan, J. Quan, T. Ramalho, A. Grabska-Barwinska, D. Hassabis, C. Clopath, D. Kumaran, and R. Hadsell. Overcoming catastrophic forgetting in neural networks. In *Proceedings of the National Academy of Sciences (PNAS)*, 2017.

[29] Francisco M Castro, Manuel J Marín-Jiménez, Nicolás Guil, Cordelia Schmid, and Karteek Alahari. End-to-end incremental learning. In *European Conference on Computer Vision (ECCV)*, pages 233–248, 2018.

[30] Yue Wu, Yinpeng Chen, Lijuan Wang, Yuancheng Ye, Zicheng Liu, Yandong Guo, and Yun Fu. Large scale incremental learning. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 374–382, 2019.

[31] Saihui Hou, Xinyu Pan, Chen Change Loy, Zilei Wang, and Dahua Lin. Learning a unified classifier incrementally via rebalancing. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2019.

[32] Giampaolo Bovenzi, Lixuan Yang, Alessandro Finamore, Giuseppe Aceto, Domenico Ciuonzo, Antonio Pescapè, and Dario Rossi. A first look at class incremental learning in deep learning mobile traffic classification. In *IFIP Traffic Monitoring Analysis (TMA)*, 2021.

[33] Marc Masana, Xialei Liu, Bartłomiej Twardowski, Mikel Menta, Andrew D Bagdanov, and Joost van de Weijer. Class-incremental learning: survey and performance evaluation on image classification. *IEEE Trans. Pattern Anal. Mach. Intell.*, 45(5):5513–5533, 2023.

[34] Sylvestre-Alvise Rebuffi, Alexander Kolesnikov, Georg Sperl, and Christoph H Lampert. iCaRL: incremental classifier and representation learning. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 2001–2010, 2017.

[35] Minsoo Kang, Jaeyoo Park, and Bohyung Han. Class-incremental learning by knowledge distillation with adaptive feature consolidation. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 16071–16080, 2022.

[36] Lloyd S Shapley. A value for n-person games. *Contributions to the Theory of Games*, 2(28):307–317, 1953.

[37] Mukund Sundararajan, Ankur Taly, and Qiqi Yan. Axiomatic attribution for deep networks. In *34th PMLR International Conference on Machine Learning (ICML)*, volume 70, pages 3319–3328, 2017.

[38] Scott M Lundberg and Su-In Lee. A unified approach to interpreting model predictions. In *30th Conference on Neural Information Processing Systems (NeurIPS)*, volume 30, pages 4765–4774, 2017.

[39] Alfredo Nascita, Antonio Montieri, Giuseppe Aceto, Domenico Ciuonzo, Valerio Persico, and Antonio Pescapé. XAI meets mobile traffic classification: Understanding and improving multimodal deep learning architectures. *IEEE Trans. Netw. Service Manag.*, 18(4):4225–4246, 2021.

[40] Meike Nauta, Jan Trienes, Shreyasi Pathak, Elisa Nguyen, Michelle Peters, Yasmin Schmitt, Jörg Schlötterer, Maurice van Keulen, and Christin Seifert. From anecdotal evidence to quantitative evaluation methods: A systematic review on evaluating explainable AI. *ACM Computing Surveys*, 55(13s):1–42, 2023.

[41] Amani Abou Rida, Rabih Amhaz, and Pierre Parrend. Metrics for evaluating interface explainability models for cyberattack detection in IoT data. In *Springer Complex Computational Ecosystems (CCE)*, 2023.

[42] Amir Mahdi Sadeghzadeh, Saeed Shiravi, and Rasool Jalili. Adversarial network traffic: Towards evaluating the robustness of deep-learning-based network traffic classification. *IEEE Trans. Netw. Service Manag.*, 18(2):

1962–1976, 2021.

[43] Chao Wang, Alessandro Finamore, Pietro Michiardi, Massimo Gallo, and Dario Rossi. Data augmentation for traffic classification. In *Springer International Passive and Active Measurement Conference (PAM)*, pages 159–186, 2024.

[44] Francesco Cerasuolo, Alfredo Nascita, Giampaolo Bovenzi, Giuseppe Aceto, Domenico Ciuonzo, Antonio Pescapè, and Dario Rossi. MEMENTO: A novel approach for class incremental learning of encrypted traffic. *Computer Networks*, page 110374, 2024.

[45] François De Keersmaeker, Yinan Cao, Gorby Kabasele Ndonda, and Ramin Sadre. A survey of public IoT datasets for network security research. *IEEE Commun. Surveys Tuts.*, 2023.

[46] Barjinder Kaur, Sajjad Dadkhah, Farzaneh Shoeleh, Euclides Carlos Pinto Neto, Pulei Xiong, Shahrear Iqbal, Philippe Lamontagne, Suprio Ray, and Ali A Ghorbani. Internet of Things (IoT) security dataset evolution: Challenges and future directions. *Internet of Things*, 22:100780, 2023.

[47] Christin Alex, Giselle Creado, Wesam Almobaideen, Orieb Abu Alghanam, and Maha Saadeh. A comprehensive survey for IoT security datasets taxonomy, classification and machine learning mechanisms. *Computers & Security*, 132:103283, 2023.

[48] Giampaolo Bovenzi, Giuseppe Aceto, Domenico Ciuonzo, Valerio Persico, and Antonio Pescapé. A hierarchical hybrid intrusion detection approach in IoT scenarios. In *IEE GLOBECOM'20*, pages 1–7, 2020.

[49] Giuseppe Aceto, Domenico Ciuonzo, Antonio Montieri, and Antonio Pescapé. Mobile encrypted traffic classification using deep learning: Experimental evaluation, lessons learned, and challenges. *IEEE Trans. Netw. Service Manag.*, 16(2):445–458, 2019.

[50] Giuseppe Aceto, Domenico Ciuonzo, Antonio Montieri, and Antonio Pescapè. MIMETIC: mobile encrypted traffic classification using multimodal deep learning. *Computer Networks*, 165:106944, 2019.

[51] Alfredo Nascita, Francesco Cerasuolo, Davide Di Monda, Jonas Thern Aberia Garcia, Antonio Montieri, and Antonio Pescapè. Machine and deep learning approaches for IoT attack classification. In *IEEE Conference on Computer Communications Workshops (INFOCOM WKSHPS)*, pages 1–6, 2022.

[52] Manuel Lopez-Martin, Belen Carro, Antonio Sanchez-Esguevillas, and Jaime Lloret. Network traffic classifier with convolutional and recurrent neural networks for Internet of Things. *IEEE Access*, 5:18042–18050, 2017.

[53] Wenbin Zhu, Xiuli Ma, Yanliang Jin, and Rui Wang. ILETC: Incremental learning for encrypted traffic classification using generative replay and exemplar. *Computer Networks*, 224:109602, 2023.

[54] Omar Abdel Wahab. Intrusion detection in the IoT under data and concept drifts: Online deep learning approach. *IEEE Internet Things J.*, 9(20): 19706–19716, 2022.